HKU Summer Course

Neural Networks and Deep Learning

Jintao Ke

Department of Civil Engineering
Faculty of Engineering
The University of Hong Kong, Pokfulam, Hong Kong
Email: kejintao@hku.hk

Outline

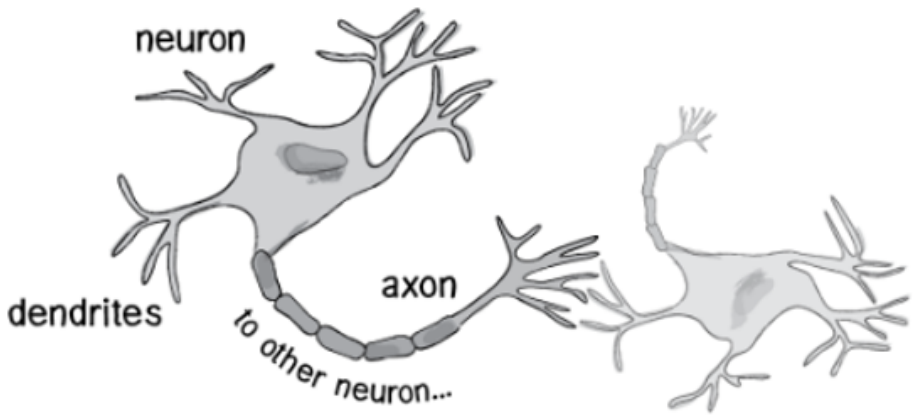
1. Introduction
2. Simple Perceptron
3. Multilayer Perceptron
4. Training Deep Neural Networks
   - Choosing Activation Functions
   - Initialization
   - Regularization
     - Early Stopping
   - Optimization Algorithms

Artificial Neural Networks

- Artificial neural network (ANN, or simply called "neural network") is a machine learning technique
- ANN is initially invented for computers to imitate how human brain works, so as to solve problems that are easy for human but difficult for computers, but nowdays ANNs can even beat humans
- ANN is a very simple implementation of local behavior observed within human brains
- ANN is a type of intelligent computing inspired by biology / nature

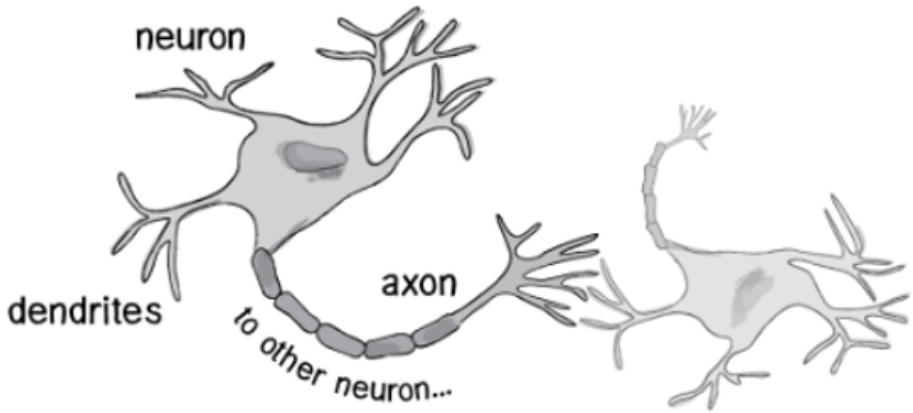
Neuron

- Human brain consists of neurons
- Neurons are individual elements in charge of storing, processing and transmitting information
- Dendrites receive signals
- Axon outputs signals
- Human brain contains approximately 86 billion neurons
- Each neuron connects to approximately 1000 other neurons



Neuron

- Human brain can be described as a biological neural network
- Neurons are interconnected to transmit elaborate patterns of signals
- Human brain processes information in a highly complex, nonlinear and parallel manner
- Human brain has the capability to learn (to improve and to deal with new problems)

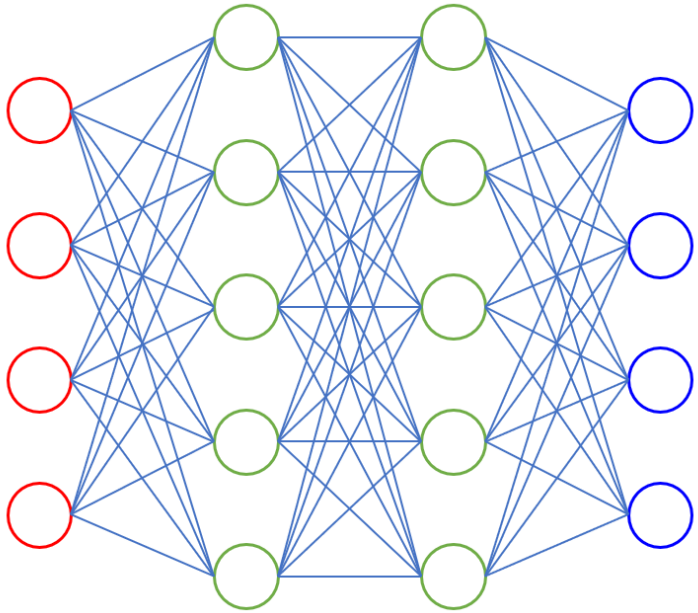


History of ANN

- Originated in 1940s. In 1943, Warren S. McCulloch, a neuroscientist, and Walter Pitts, a logician, showed that networks of artificial neurons can, in principle, compute any arithmetic or logical function
- In 1958, Frank Rosenblatt invented perceptron, an ANN consist of a single neuron which can learn
- 1970s: AI winter; weak computation capacity
- 1980s-1990s: backpropagation is widely used
- 1990s-2000s: second AI winter; performance worse than other machine learning algorithms
- 2010-: deep learning; powerful in big data

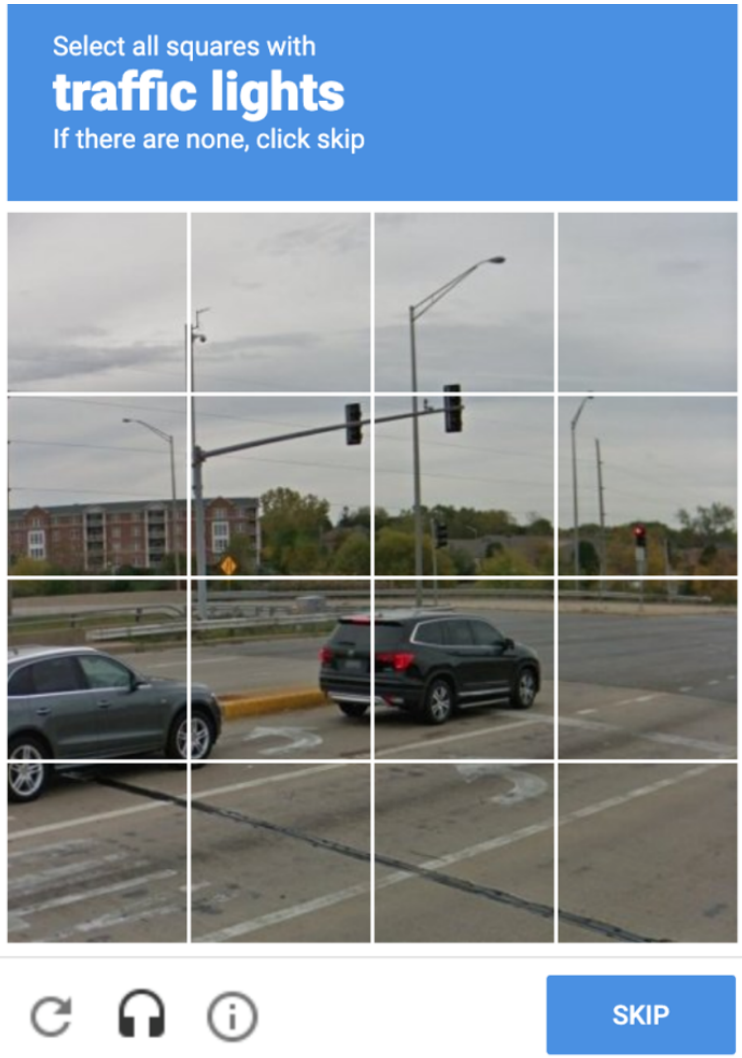
Structures of ANN

- ANN is a network of neurons (nodes)
- Nodes are connected to each other
- Each node takes many inputs
- Information flow through the network and is processed in parallel
- Each link has a weight
- Each node executes simple calculations but the network as a whole can complete complex tasks
- ANN learns by adjusting the weights and (sometimes) changing its internal structure, based on the output



Applications of ANN

- Tasks easy for human but difficult for machines
- Pattern Recognition – probably the most common applications
  - Tell cats from dogs
  - Character recognition (license plate, paper book)
  - Facial / fingerprint recognition
- Signal Processing – filter noise and amplify sound for hearing aids
- Control – self-driving cars



https://sites.google.com/view/kejintao

Applications of ANN

- Soft Sensor & Data Fusion – process many measurements together
  - Diagnose disease (age, gender, life style, test results, etc.)
  - Predict traffic congestion (network structure, traffic volume, working day, time, weather, etc.)
- Time Series Prediction
  - Will the stock price rise or fall tomorrow?
  - Will it rain or not in an hour?
- Anomaly Detection – identify something that doesn't fit the pattern
  - Credit card fraud
  - Junk mail
  - Heart attack
  - Writing style

Artificial Neural Networks

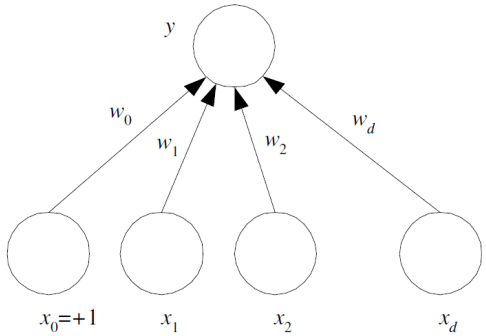
- Cognitive scientists and neuroscientists: the aim is to understand the functioning of the brain by building models of the natural neural networks.
- Machine learning researchers: the aim (more pragmatic) is to build better computer systems based on inspirations from studying the brain.
- A human brain has:
  - Large number (10^{11}) of neurons as processing units
  - Large number (10^{4}) of synapses per neuron as memory units
  - Parallel processing capabilities
  - Distributed computation/memory
  - High robustness to noise and failure
- Artificial neural networks (ANN) mimic some characteristics of the human brain, especially with regard to the computational aspects.

Dataset and Problem (Revisit)

- Training set with N samples: $\mathcal{X} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$ (here we use $y^{(n)}$ instead of y_n since neural networks are very complex).
- Regression problem: Use the training set to find out the set of parameters or weights $\hat{\mathbf{w}}$ that makes $g(\mathbf{x}^{(n)}|\mathbf{w})$ predict $y^{(n)}$ as accurately as possible.
- Classification problem: Use $\mathcal{X}$ to train a decision rule to classify a sample $\mathbf{x}$ into either a positive or negative sample, based on the discriminant functions $g(\mathbf{x}^{(n)}|\mathbf{w})$, for example,

$$\hat{y}_n = \begin{cases} 1, & \text{if } g(\mathbf{x}^{(n)}|\mathbf{w}) > 0 \\ 0, & \text{if } g(\mathbf{x}^{(n)}|\mathbf{w}) < 0 \end{cases} \tag{1}$$

Perceptron



- The output y is a weighted sum of the inputs $\mathbf{x} = (x_0, x_1, x_2, ..., x_D)^T$:

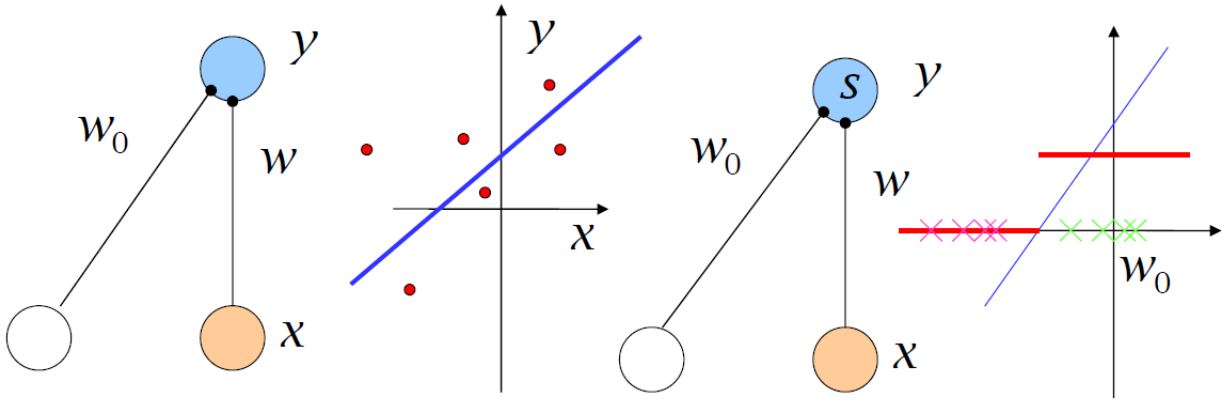
$$y = \sum_{j=1}^{D} w_j x_j + w_0 = \mathbf{w}^T \mathbf{x} \tag{2}$$

where x_0 is a special bias unit with $x_0 = 1$.

- w_1,...,w_D are called weights (like the coefficients in linear regression) and w_0 is called bias (like the intercept). $\mathbf{w} = (w_0, \cdots, w_D)^T$ are called the connection weights.

What a Perceptron Does

- Regression vs. classification:



- For classification, To implement a linear discriminant function, we need the threshold function

$$s(a) = \begin{cases} 1, & \text{if } a > 0 \\ 0, & \text{if } a \leq 0 \end{cases} \tag{3}$$

to define the following decision rule:

$$\text{Choose} \begin{cases} C_1, & \text{if } s(\mathbf{w}^T\mathbf{x}) = 1 \\ C_2, & \text{if } s(\mathbf{w}^T\mathbf{x}) = 0 \end{cases} \tag{4}$$

Sigmoid Function

- Instead of using the threshold function to give a discrete output in $\{0, 1\}$, we may use the sigmoid function

$$\text{sigmoid}(a) = \frac{1}{1 + \exp(-a)} = \frac{\exp(a)}{1 + \exp(a)} \tag{5}$$

to give a continuous real-valued output in $(0, 1)$:

$$y = \text{sigmoid}(\mathbf{w}^T \mathbf{x}) \tag{6}$$

where $\exp(a) = e^a$

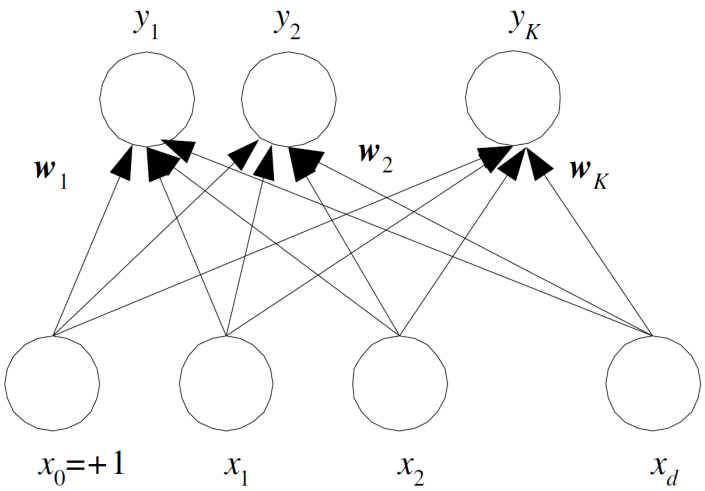
- The output may be interpreted as the probability that the input $\mathbf{x}$ belongs to C_1
- What if we have $K > 2$ classes?

Sigmoid Function for $K > 2$ Outputs

- K perceptrons, each with a weight vector $\mathbf{w}_i$:

$$y_i = \sum_{j=1}^{D} w_{ij} x_j + w_{i0} = \mathbf{w}_i^T \mathbf{x} \quad \text{or} \quad \mathbf{y} = \mathbf{W}\mathbf{x} \tag{7}$$

where w_{ij} is the weight from input x_j to output y_i and each row of the $K \times (D+1)$ matrix $\mathbf{W}$ is the weight vector of one perceptron $\mathbf{w}_i$.



Classification

- Classification rule:

$$\text{Choose } C_i \text{ if } y_i = \max_k y_k \tag{8}$$

That is, each class corresponds to an output perceptron, and we choose the class with the maximum output.

- If we need to express the probabilities of $\mathbf{x}$ belonging to each class, we can use softmax function to define y_i as follows:

$$y_i = \text{softmax}\left(\mathbf{w}_i^T \mathbf{x}^{(n)}\right) = \frac{\exp\left(\mathbf{w}_i^T \mathbf{x}\right)}{\sum_{k=1}^{K} \exp\left(\mathbf{w}_k^T \mathbf{x}\right)} \tag{9}$$

Sigmoid and Softmax

- Derivative of sigmoid:

$$y = \text{sigmoid}(a) = \frac{1}{1 + \exp(-a)} \tag{10}$$

$$\frac{\mathrm{d}y}{\mathrm{d}a} = \frac{1}{(1 + \exp(-a))^2} \exp(-a) = y(1 - y) \tag{11}$$

- Derivative of softmax:

$$y_i = \text{softmax}(a_i) = \frac{\exp(a_i)}{\sum_k \exp(a_k)} \tag{12}$$

$$\frac{\partial y_i}{\partial a_j} = y_i(\delta_{ij} - y_j) \tag{13}$$

where $\delta_{ij} = 1$ if $i = j$, and $\delta_{ij} = 0$ if $i \neq j$

Parameter Learning

- Task: Find weights to minimize the error function $E(\mathbf{w}|\mathcal{X})$, where $\mathcal{X} = \{\mathbf{x}^{(n)}, y^{(n)}\}, \forall n \in \{1, ..., N\}$

$$\arg\min_{\mathbf{w}} E(\mathbf{w}|\mathcal{X}) \tag{14}$$

- Recall that for regression problem, the mean squared error function:

$$E(\mathbf{w}|\mathcal{X}) = \frac{1}{2}\sum_{n=1}^{N}\left[y^{(n)} - \mathbf{w}^T\mathbf{x}^{(n)}\right]^2 \tag{15}$$

- For binary classification problem, the cross-entropy error function:

$$E(\mathbf{w}|\mathcal{X}) = -\sum_{n=1}^{N}\left[y^{(n)}\log z^{(n)} + (1 - y^{(n)})\log(1 - z^{(n)})\right] \tag{16}$$

where $z^{(n)} = \text{sigmoid}\left(\mathbf{w}^T\mathbf{x}^{(n)}\right)$

Parameter Learning (2)

- For $K > 2$ classification problem, the cross-entropy error function:

$$E(\{\mathbf{w}_i\}|\mathcal{X}) = -\sum_{n=1}^{N}\sum_{i=1}^{K} y_i^{(n)} \log z_i^{(n)} \tag{17}$$

where $z_i^{(n)} = \text{softmax}\left(\mathbf{w}_i^T \mathbf{x}^{(n)}\right) = \frac{\exp\left(\mathbf{w}_i^T \mathbf{x}^{(n)}\right)}{\sum_{k=1}^{K} \exp\left(\mathbf{w}_k^T \mathbf{x}^{(n)}\right)}$

- To summarize, the error functions have the following form:

$$E(\mathbf{w}|\mathcal{X}) = \sum_{n=1}^{N} \mathcal{L}\left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)}\right) \tag{18}$$

- Algorithm for solving the Perceptron: Stochastic gradient descent (SGD)

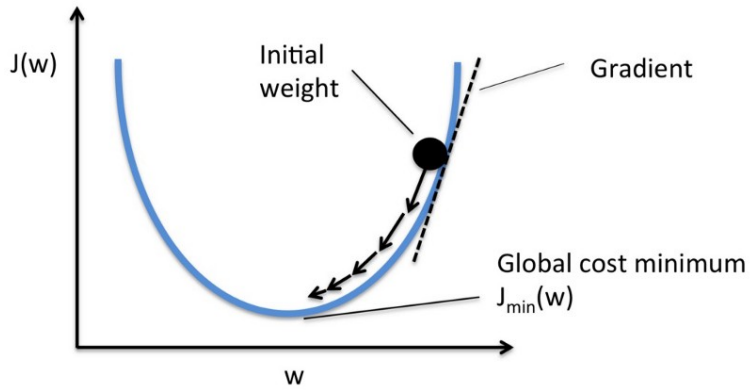
Gradient Descent (Revisit)

- A numerical method for minimizing an error function (also called loss function, cost function)
  1. Initialize $\mathbf{w}$
  2. Repeat until convergence

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} E(\mathbf{w}|\mathcal{X}) \tag{19}$$

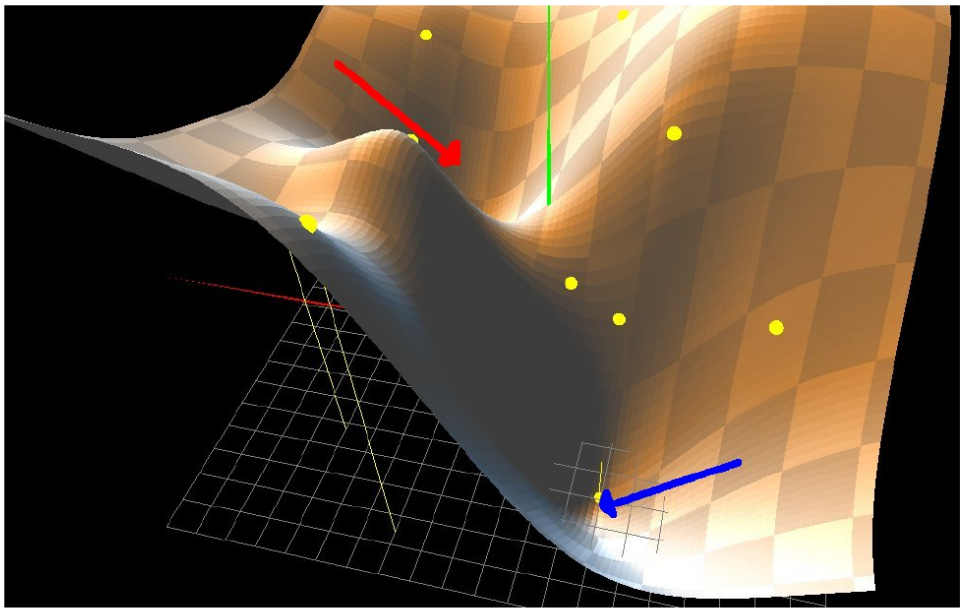
where $\nabla_{\mathbf{w}} E(\mathbf{w}|\mathcal{X}) = \left(\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_D} \right)^T$

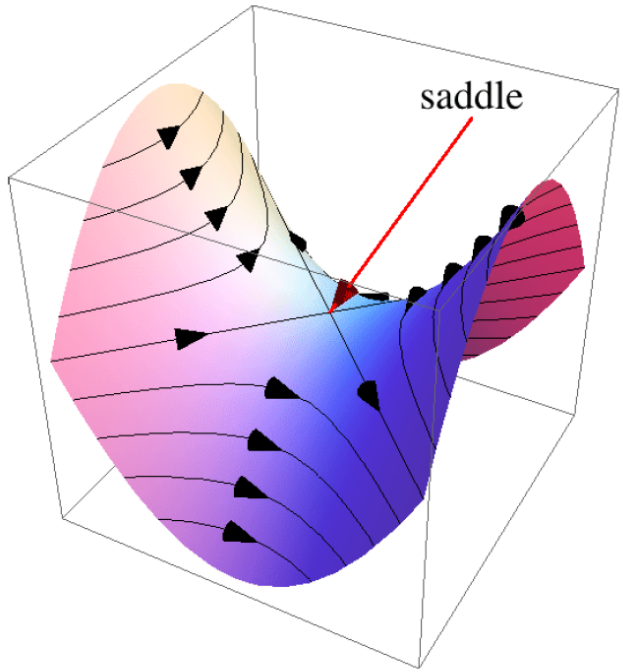
- The learning rate α can change from iteration to iteration.



Gradient Descent (Revisit)

- Gradient descent can get stuck at local minima or saddle points





- Nevertheless, it usually works well.

Stochastic Gradient Descent

- For given N training samples, the error function for all samples can be written as:

$$J(\mathbf{w}) = E(\mathbf{w}|\mathcal{X}) = \sum_{n=1}^{N} \mathcal{L}\left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)}\right) \tag{20}$$

- To get $\nabla_{\mathbf{w}} J(\mathbf{w})$, we need to calculate the gradient for each sample:

$$\nabla_{\mathbf{w}} \mathcal{L}\left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)}\right) \tag{21}$$

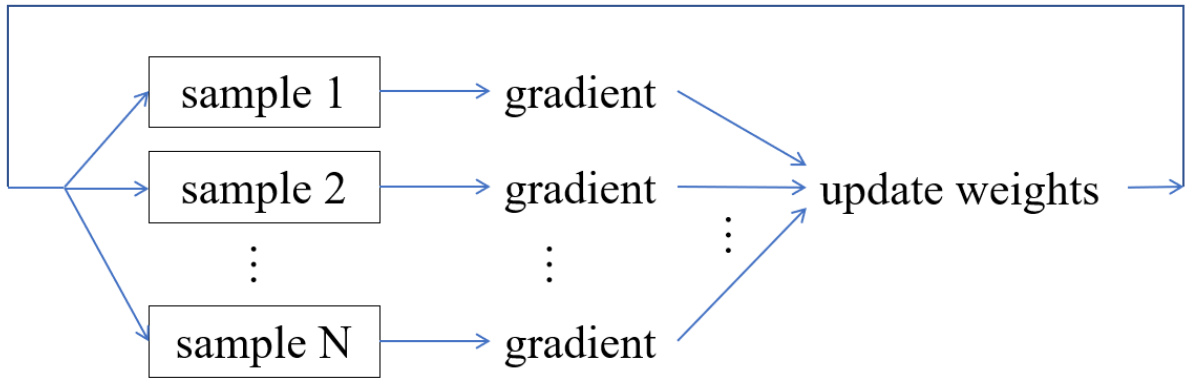
  It is time consuming when the sample size N is very large (millions, billions, ...)

- Stochastic Gradient Descent (SGD): At each iteration, estimate the gradient $\nabla_{\mathbf{w}} J(\mathbf{w})$ based on a randomly sampled subset of examples, which is called a minibatch

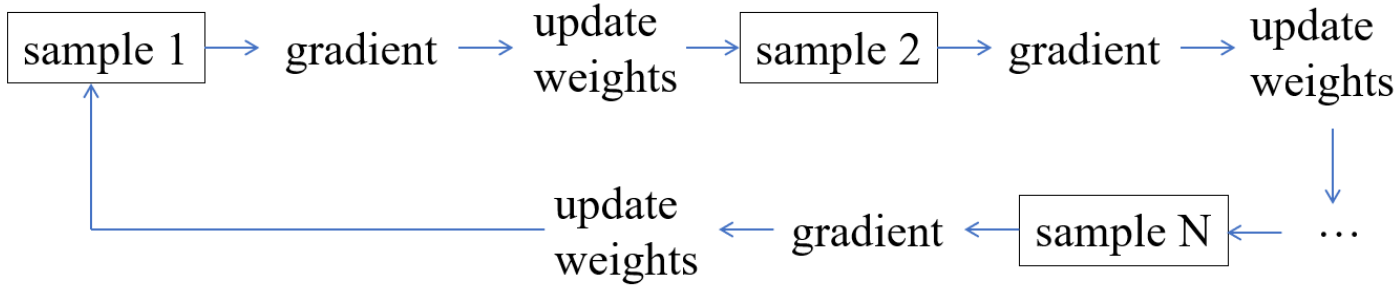
Stochastic Gradient Descent (2)

- Advantages of Stochastic Gradient Descent
  - No need to store the whole sample.
  - the parameters are adjusted a little bit at each iteration to reduce the error without forgetting what was learned previously.

Batch GD

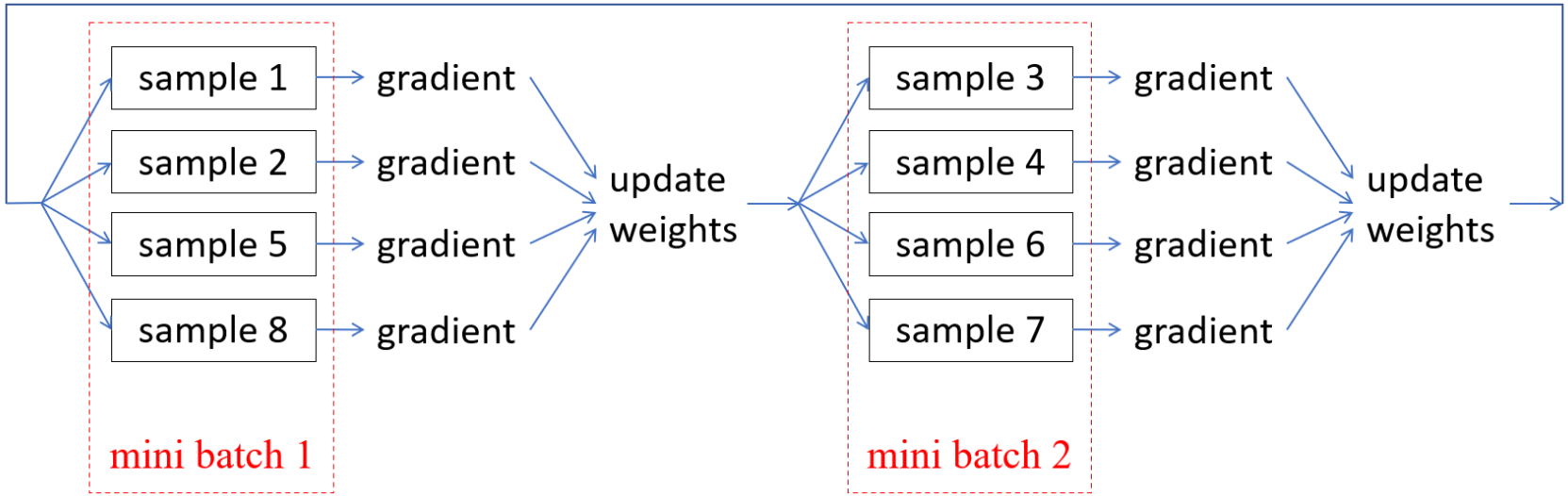


Stochastic GD



Stochastic Gradient Descent (3)

- In Stochastic Gradient Descent with minibatch (called minibatch GD), the total N samples are divided into B mini batches
  - If $B = 1$: minibatch GD = batch GD
  - If $B = N$: minibatch GD = stochastic GD
- Example: suppose the training set has 8 samples, which are divided into two mini batches



Stochastic Gradient Descent (4)

- Algorithm: Minibatch GD
  1. Initialize $\mathbf{w}$
  2. Repeat until convergence
     1. Sample a minibatch of m samples $((\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)}))$ from the training set
     2. Compute the gradient: $\Delta\mathbf{w} \leftarrow \frac{1}{m}\sum_{n=1}^{m} \nabla_{\mathbf{w}} \mathcal{L}\left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)}\right)$
     3. Update parameters: $\mathbf{w} \leftarrow \mathbf{w} - \eta\Delta\mathbf{w}$, where η is the learning rate.
- Then the question is how to compute the gradient for each sample: $\nabla_{\mathbf{w}} \mathcal{L}\left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)}\right)$?
- Answer: for simple perceptron, we can directly derive the update rule by calculating the gradient for each sample.

SGD for Regression

- The error on a single sample $(\mathbf{x}^{(n)}, y^{(n)})$:

$$E^{(n)}(\mathbf{w}|\mathbf{x}^{(n)}, y^{(n)}) = \frac{1}{2}(y^{(n)} - \hat{y}^{(n)})^2 = \frac{1}{2}\left[y^{(n)} - \mathbf{w}^T\mathbf{x}^{(n)}\right]^2 \tag{22}$$

where $y^{(n)}$ is the true label and $\hat{y}^{(n)}$ is the estimated label of sample n.

- This gives the gradient for each sample:

$$\Delta w_j^{(n)} = \frac{\partial E^{(n)}(\mathbf{w}|\mathbf{x}^{(n)}, y^{(n)})}{\partial w_j} = -\left(y^{(n)} - \hat{y}^{(n)}\right)x_j^{(n)} \tag{23}$$

and the following online update rule:

$$w_j \leftarrow w_j - \eta\Delta w_j^{(n)} = w_j + \eta\left(y^{(n)} - \hat{y}^{(n)}\right)x_j^{(n)} \tag{24}$$

where the learning rate η usually decreases gradually over time to facilitate convergence.

SGD for Binary Classification

- Likelihood of a single sample $(\mathbf{x}^{(n)}, y^{(n)})$:

$$L = (\hat{y}^{(n)})^{y^{(n)}} (1 - \hat{y}^{(n)})^{1-y^{(n)}} \tag{25}$$

where the estimated output $\hat{y}^{(n)} = \text{sigmoid}(\mathbf{w}^T \mathbf{x}^{(n)}) = \frac{\exp\left(\mathbf{w}^T \mathbf{x}^{(n)}\right)}{1+\exp\left(\mathbf{w}^T \mathbf{x}^{(n)}\right)}$.

- Cross-entropy error function:

$$E^{(n)}(\mathbf{w}|\mathbf{x}^{(n)}, y^{(n)}) = -\log L = -y^{(n)} \log \hat{y}^{(n)} - (1 - y^{(n)}) \log(1 - \hat{y}^{(n)}) \tag{26}$$

- This gives the gradient for each sample:

$$\Delta w_j^{(n)} = \frac{\partial E^{(n)}(\mathbf{w}|\mathbf{x}^{(n)}, y^{(n)})}{\partial w_j} = -\left(y^{(n)} - \hat{y}^{(n)}\right) x_j^{(n)} \tag{27}$$

and the following online update rule:

$$w_j \leftarrow w_j - \eta \Delta w_j^{(n)} = w_j + \eta \left(y^{(n)} - \hat{y}^{(n)}\right) x_j^{(n)} \tag{28}$$

SGD for Multi-output Classification $K > 2$

- Likelihood of a single sample $(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})$, where $y_i^{(n)} = 1$ if $\mathbf{x}^{(n)} \in C_i$ and 0 otherwise:

$$L = \prod_{i=1}^{K} (\hat{y}_i^{(n)})^{y_i^{(n)}} \tag{29}$$

where the estimated output

$$\hat{y}_i^{(n)} = \text{softmax}(\mathbf{w}_i^T \mathbf{x}^{(n)}) = \frac{\exp\left(\mathbf{w}_i^T \mathbf{x}^{(n)}\right)}{\sum_{k=1}^{K} \exp\left(\mathbf{w}_k^T \mathbf{x}^{(n)}\right)} \tag{30}$$

- Cross-entropy error function:

$$E^{(n)}(\{\mathbf{w}_i\}|\mathbf{x}^{(n)}, y^{(n)}) = -\log L = -\sum_{i=1}^{K} y_i^{(n)} \log \hat{y}_i^{(n)} \tag{31}$$

SGD for Multi-output Classification $K > 2$

- Gradient of the weight of the jth feature, mth class and nth sample:

$$\Delta w_{mj}^{(n)} = \frac{\partial E}{\partial w_{mj}} = -\sum_{i=1}^{K} \frac{y_i^{(n)}}{\hat{y}_i^{(n)}} \frac{\partial \hat{y}_i^{(n)}}{\partial w_{mj}} = -\sum_{i=1}^{K} \frac{y_i^{(n)}}{\hat{y}_i^{(n)}} \frac{\partial \hat{y}_i^{(n)}}{\partial a_m^{(n)}} \frac{\partial a_m^{(n)}}{\partial w_{mj}} \tag{32}$$

- where $\frac{\partial a_m^{(n)}}{\partial w_{mj}} = x_j^{(n)}$ and $\frac{\partial \hat{y}_i^{(n)}}{\partial a_m^{(n)}} = \hat{y}_i^{(n)}(\delta_{im} - \hat{y}_m^{(n)})$
- Then we have:

$$\begin{aligned} \Delta w_{mj}^{(n)} &= -\sum_{i=1}^{K} y_i^{(n)} (\delta_{im} - \hat{y}_m^{(n)}) x_j^{(n)} \\ &= -\left[\sum_{i=1}^{K} y_i^{(n)} \delta_{im} - \hat{y}_m^{(n)} \sum_{i=1}^{K} y_i^{(n)} \right] x_j^{(n)} \\ &= -\left(y_m^{(n)} - \hat{y}_m^{(n)} \right) x_j^{(n)} \end{aligned} \tag{33}$$

SGD for Multi-output Classification $K > 2$

- Without loss of generality, the gradient for the nth sample is:

$$\Delta w_{ij}^{(n)} = \frac{\partial E^{(n)}(\{\mathbf{w}_i\}|\mathbf{x}^{(n)}, y^{(n)})}{\partial w_{ij}} = -\left(y_i^{(n)} - \hat{y}_i^{(n)}\right) x_j^{(n)} \tag{34}$$

- Online update rule:

$$w_{ij} \leftarrow w_{ij} - \eta \Delta w_{ij}^{(n)} = w_{ij} + \eta \left(y_i^{(n)} - \hat{y}_i^{(n)}\right) x_j^{(n)} \tag{35}$$

$\forall i \in \{1, ..., K\}, j \in \{1, ..., D\}$

- To summarize, for either mean squared error (regression) or cross-entropy error (classification), the update rule is:

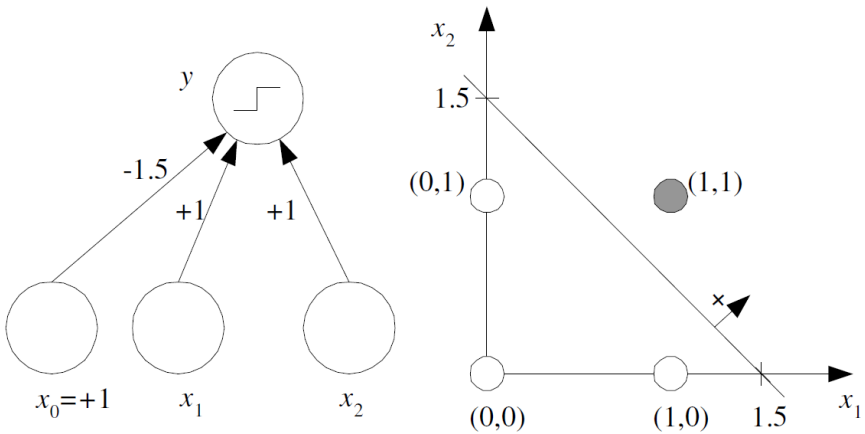
$$\text{Update} = \text{LearningRate} \times (\text{TrueLabel} - \text{EstimatedLabel}) \times \text{Input} \tag{36}$$

Learning Boolean AND

- Learning a Boolean function is a two-class classification problem.
- AND function with 2 inputs and 1 output:

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

- Perceptron for AND and its geometric interpretation:



Learning Boolean XOR

- A simple perceptron can only learn linearly separable Boolean functions such as AND and OR but not linearly nonseparable functions such as XOR:

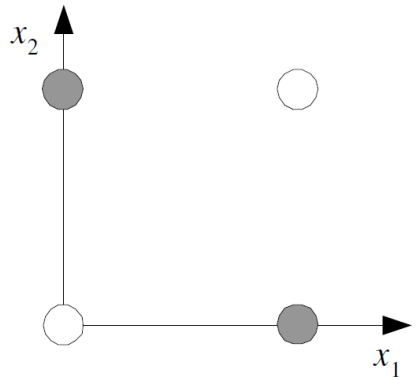
$$x_1 \text{ XOR } x_2 = \begin{cases} 1, & \text{if } x_1 = 0, x_2 = 1; \text{ or } x_1 = 1, x_2 = 0 \\ 0, & \text{if } x_1 = 1, x_2 = 1; \text{ or } x_1 = 0, x_2 = 0 \end{cases} \tag{37}$$

- XOR function with 2 inputs and 1 output:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Learning Boolean XOR (2)

- Is it possible to find a linear function $y = w_0 + w_1 x_1 + w_2 x_2$ to separate the positive and negative samples of XOR?

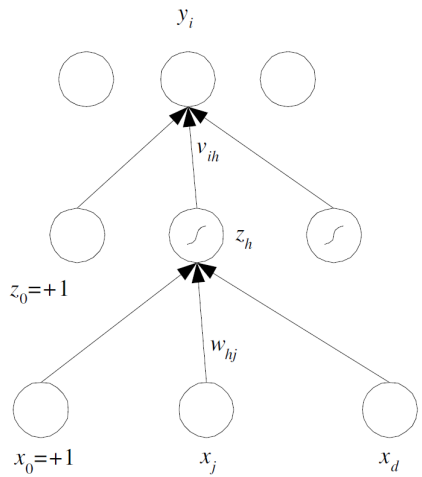


- There do not exist w_0, w_1, w_2 that satisfy the following inequalities:

$$\begin{aligned} y|_{x_1=0,x_2=0} &= w_0 \leq 0 \\ y|_{x_1=0,x_2=1} &= w_0 + w_2 > 0 \\ y|_{x_1=1,x_2=0} &= w_0 + w_1 > 0 \\ y|_{x_1=1,x_2=1} &= w_0 + w_1 + w_2 \leq 0 \end{aligned} \tag{38}$$

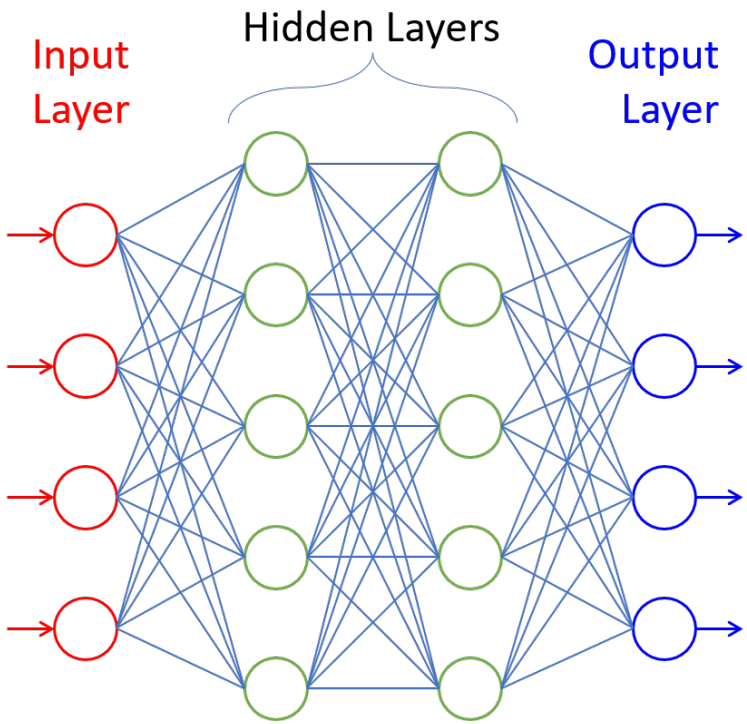
Multilayer Perceptrons

- A multilayer perceptron (MLP) has a hidden layer between the input and output layers.
- MLP can implement nonlinear discriminant functions (for classification) and nonlinear regression functions (for regression).
- MLP is a classical type of artificial neural networks.
- We call this a two-layer network because the input layer performs no computation.



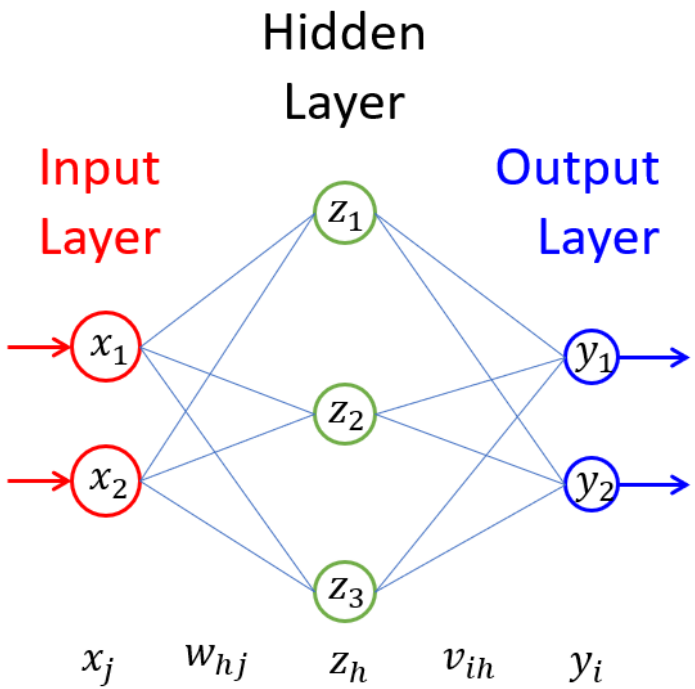
Multilayer Perceptrons

- An input layer, an output layer, one or more hidden layers
- Different layers can have different numbers of neurons
- Fully connected neural networks: each node (neuron) is connected to all the nodes (neurons) in the next layer



Forward Propagation

- Consider a two-layer network: input layer with D dimensions (features), hidden layer with H dimensions, and output layer with K dimensions.
  - Input units: $x_j, \forall j \in \{1, ..., D\}$
  - Hidden units: $z_h, \forall h \in \{1, ..., H\}$
  - Output units: $y_i, \forall i \in \{1, ..., K\}$



Forward Propagation (2)

- Input-to-hidden:

$$z_h = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x}) = \frac{1}{1 + \exp\left[-\left(\sum_{j=1}^{D} w_{hj} x_j + w_{h0}\right)\right]} \tag{39}$$

  - The hidden units must implement a nonlinear function (e.g., sigmoid or other functions), otherwise it has no difference with a simple perceptron.
  - this nonlinear function is called activation function.

- Hidden-to-output:

$$y_i = \mathbf{v}_i^T \mathbf{z} = \sum_{h=1}^{H} v_{ih} z_h + v_{i0} \tag{40}$$

  - Regression: a linear output unit
  - Classification: a sigmoid unit (for binary classification with $K = 2$) or K output units with softmax (for multi-class classification with $K > 2$)

Forward Propagation (3)

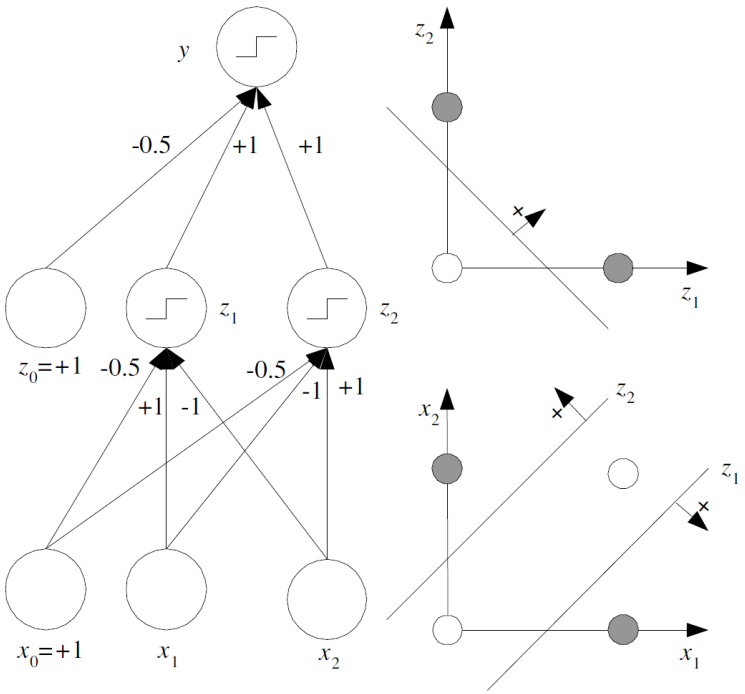
- The hidden units make a nonlinear transformation from the D-dimensional input space to the H-dimensional space spanned by the hidden units.
- In the new H-dimensional space, the output layer implements a linear function.
- Multiple hidden layers may be used for implementing more complex functions of the inputs, but learning the network weights in such deep networks will be more complicated (to be considered later in the topic of Deep Learning Models).

MLP for XOR

- Recall the XOR operation:

$$x_1 \text{ XOR } x_2 = \begin{cases} 1, & \text{if } x_1 = 0, x_2 = 1; \text{ or } x_1 = 1, x_2 = 0 \\ 0, & \text{if } x_1 = 1, x_2 = 1; \text{ or } x_1 = 0, x_2 = 0 \end{cases} \tag{41}$$

- It can be implemented by a MLP with one hidden layer.



MLP as a Universal Approximator

- The result for arbitrary Boolean functions can be extended to the continuous case.
- Universal approximation theorem: An MLP with one hidden layer can approximate any well-behaved function, under mild assumptions on the activation function, given sufficiently many hidden units.

Stochastic Gradient Descent for MLP

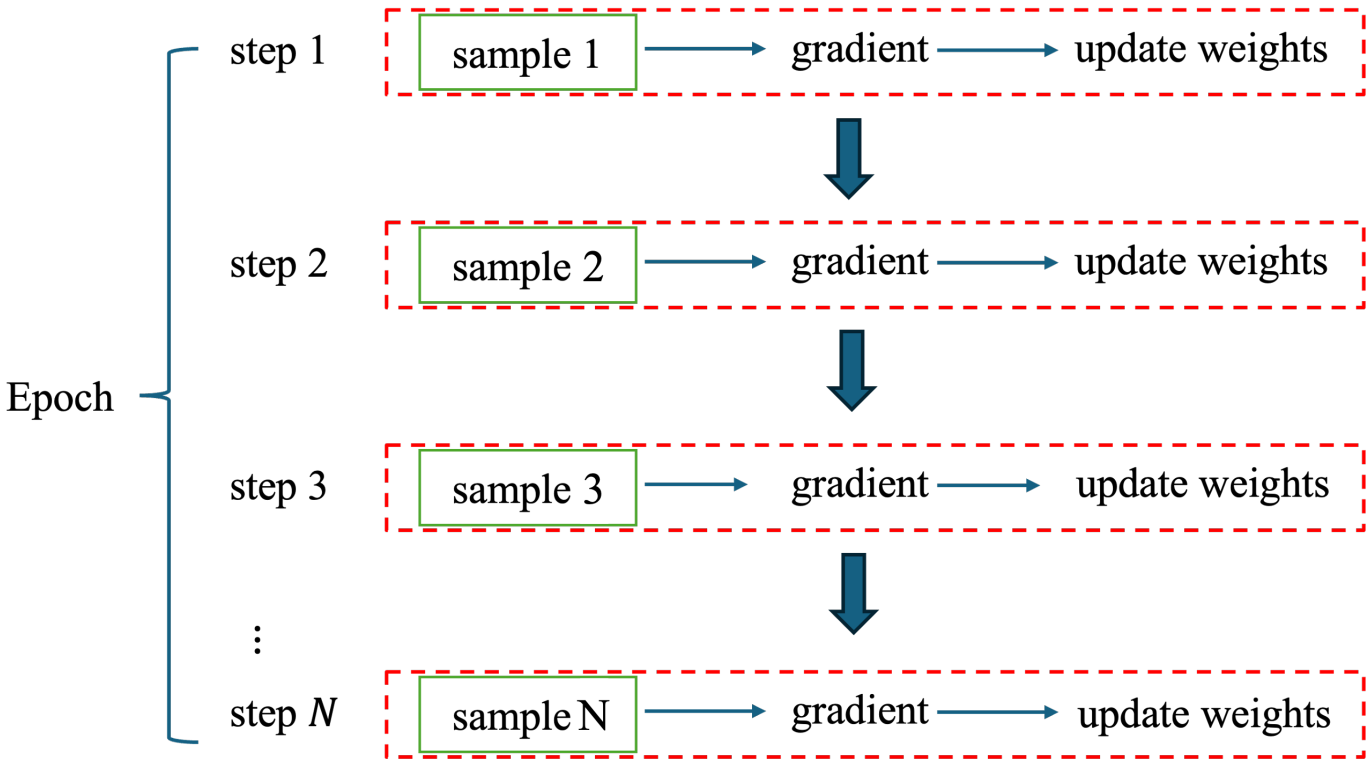
- How to learn the weights $\mathbf{w}$ (including hidden-to-output weights $\mathbf{v}$ and input-to-hidden weights $\mathbf{w}$ in the two-layer network case) of MLP?
- Algorithm: Minibatch GD
  1. Initialize $\mathbf{w}$
  2. Repeat until convergence
     1. Sample a minibatch of m samples $((\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), ..., (\mathbf{x}^{(m)}, y^{(m)}))$ from the training set
     2. Compute the gradient: $\Delta \mathbf{w} \leftarrow \frac{1}{m} \sum_{n=1}^{m} \nabla_{\mathbf{w}} \mathcal{L} \left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)} \right)$
     3. Update parameters: $\mathbf{w} \leftarrow \mathbf{w} - \eta \Delta \mathbf{w}$, where η is the learning rate.
- How to compute the gradient: $\nabla_{\mathbf{w}} \mathcal{L} \left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)} \right)$?
- Answer: Backpropagation

Epoch vs Step

- Epoch: An epoch refers to one complete pass through the **entire training dataset**.
  For example, if you have 10,000 training samples and you're running 1 epoch, the model will see all 10,000 samples once.
- Step: Steps typically refer to the number of batches processed during training.
  Each batch contains a subset of your data, and the model is updated after processing each batch. A step is equivalent to processing one batch of data and updating the model weights based on that batch.
- Example: If you set a batch size of 100, and you have 10,000 training samples, it will take 100 steps to complete 1 epoch (i.e., 10,000 / 100 = 100 steps).

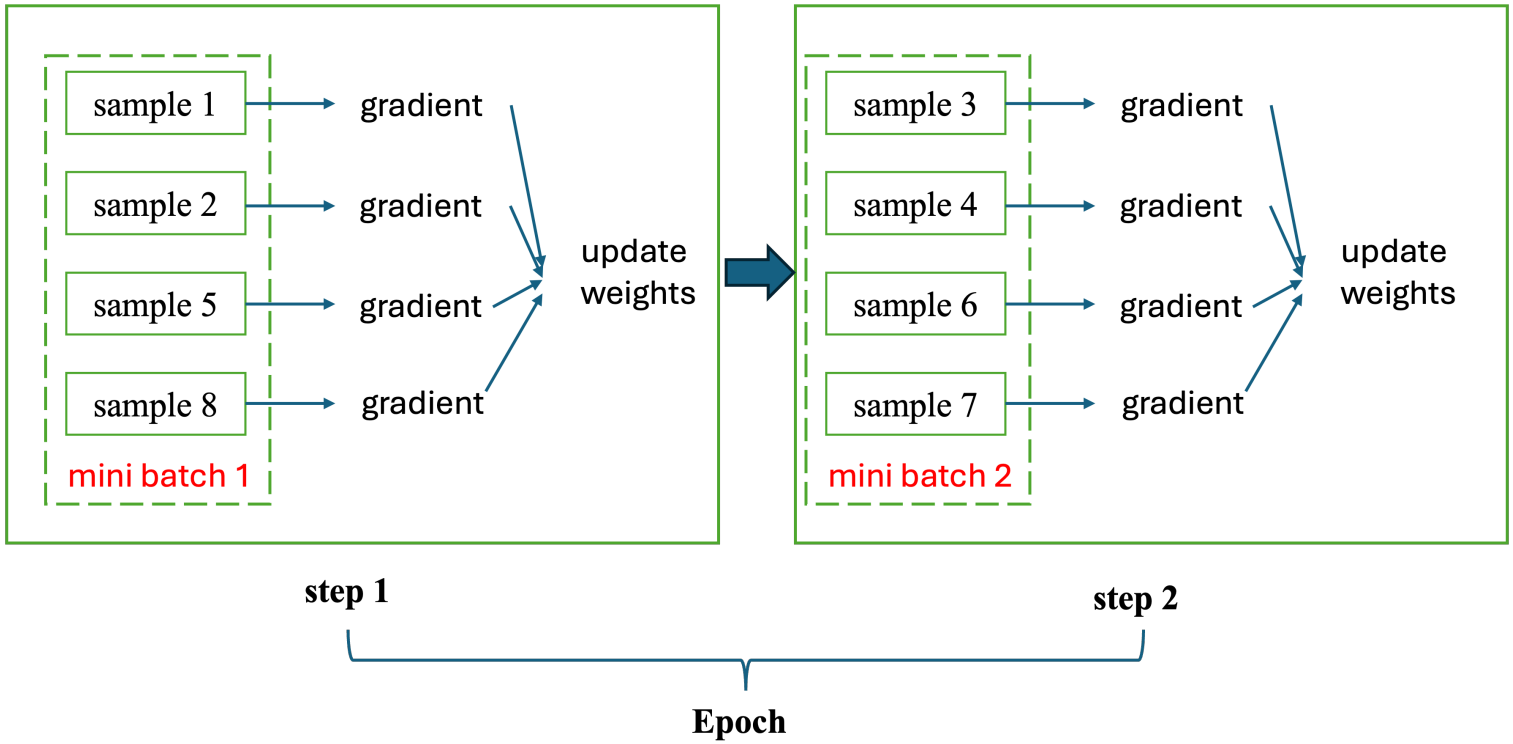
Epoch vs Step (Cont'd)

- For SGD, the batch size is 1. Suppose the training dataset contains N samples, and then we have $N/1 = N$ steps for a complete 1 epoch. The illustration is as follows:



Epoch vs Step (Cont'd)

- For Minibatch GD, suppose the training set has 8 samples, which are divided into two mini-batches. The training process for each mini-batch represents one step. The training process for all batches represents one epoch. The illustration is as follows:



Chain Rule

- Suppose $y = h(z), z = g(x)$, then:

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}y}{\mathrm{d}z}\frac{\mathrm{d}z}{\mathrm{d}x} \tag{42}$$

- Example: $g(x) = x^2$, $h(z) = \sin z$, $y = h(g(x)) = \sin(x^2)$, then:

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}y}{\mathrm{d}z}\frac{\mathrm{d}z}{\mathrm{d}x} = \cos(x^2) \cdot 2x \tag{43}$$

Chain Rule for Gradients

- Let $f(z_1, ..., z_n)$ be a function of n variables, then the gradient:

$$\nabla_{(z_1,...,z_n)} f = \left(\frac{\partial f}{\partial z_1}, \frac{\partial f}{\partial z_2}, ..., \frac{\partial f}{\partial z_n} \right)^T \tag{44}$$

- If each z_i is a function of $x_1, ..., x_D$, i.e., $z_i = g_i(x_1, ..., x_D)$, then f is also a function of $x_1, ..., x_D$ and the gradient:

$$\nabla_{(x_1,...,x_D)} f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_D} \right)^T \tag{45}$$

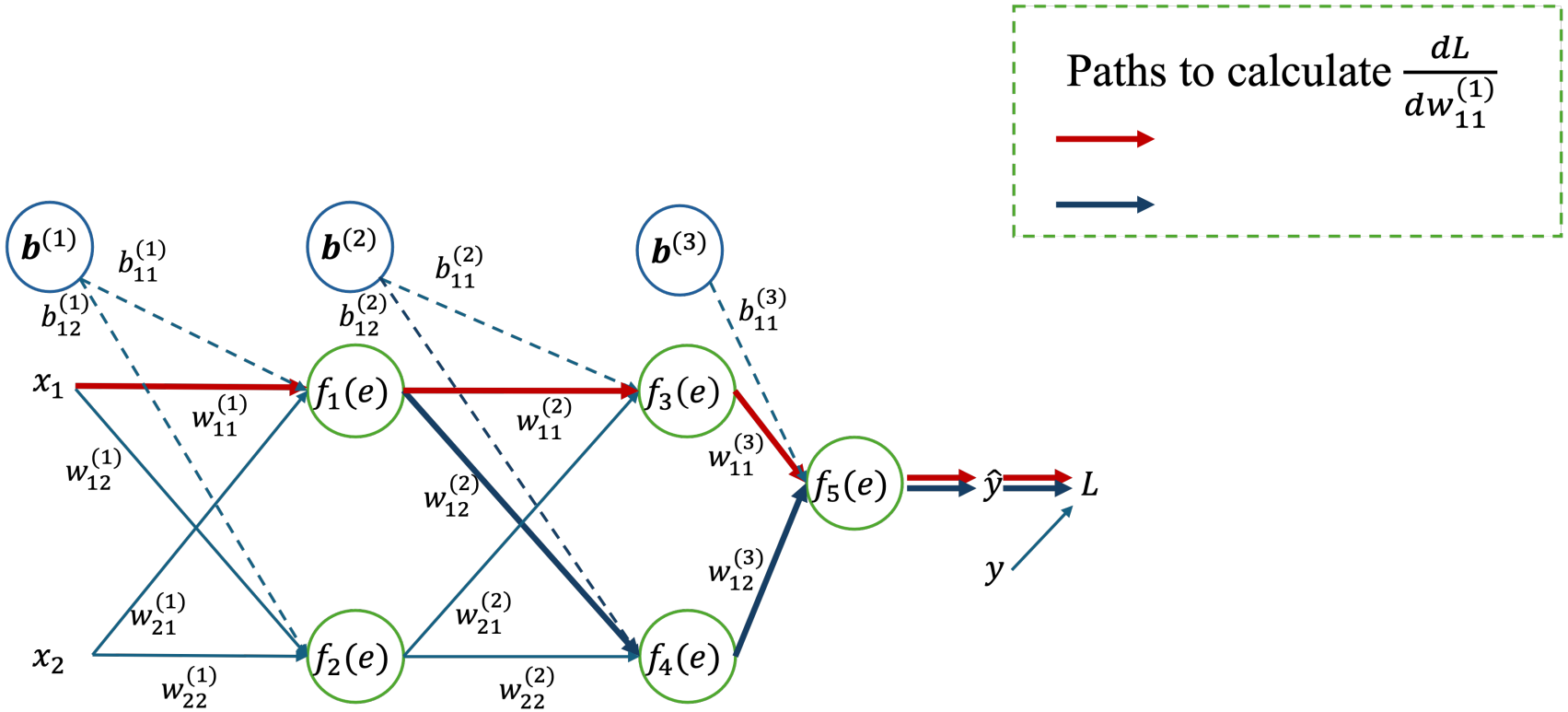
where

$$\frac{\partial f}{\partial x_j} = \sum_i \frac{\partial f}{\partial g_i} \frac{\partial g_i}{\partial x_j} \tag{46}$$

Chain Rule for Gradients (Cont'd)

- Example: Suppose y is the true label, $\hat{y}$ is the predicted label. The loss L can be calculated by MSE as follows:

$$L = \frac{1}{2}(y - \hat{y})^2 \tag{47}$$



Chain Rule for Gradients (Cont'd)

$$\begin{aligned}\frac{\partial L}{\partial w_{11}^{(1)}} &= \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial f_5(e)} \left(\frac{\partial f_5(e)}{\partial f_3(e)} \frac{\partial f_3(e)}{\partial f_1(e)} + \frac{\partial f_5(e)}{\partial f_4(e)} \frac{\partial f_4(e)}{\partial f_1(e)} \right) \frac{\partial f_1(e)}{\partial w_{11}^{(1)}} \\ &= -(y - \hat{y}) \left(w_{11}^{(3)} w_{11}^{(2)} + w_{21}^{(3)} w_{12}^{(2)} \right) w_{11}^{(1)} \end{aligned} \tag{48}$$

Suppose that function $f_1(e), f_2(e), f_3(e), f_4(e)$, and $f_5(e)$ compute the sum of their inputs and return the result as their output.

$$\mathbf{w}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \mathbf{w}^{(2)} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} \\ w_{21}^{(2)} & w_{22}^{21)} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix},$$

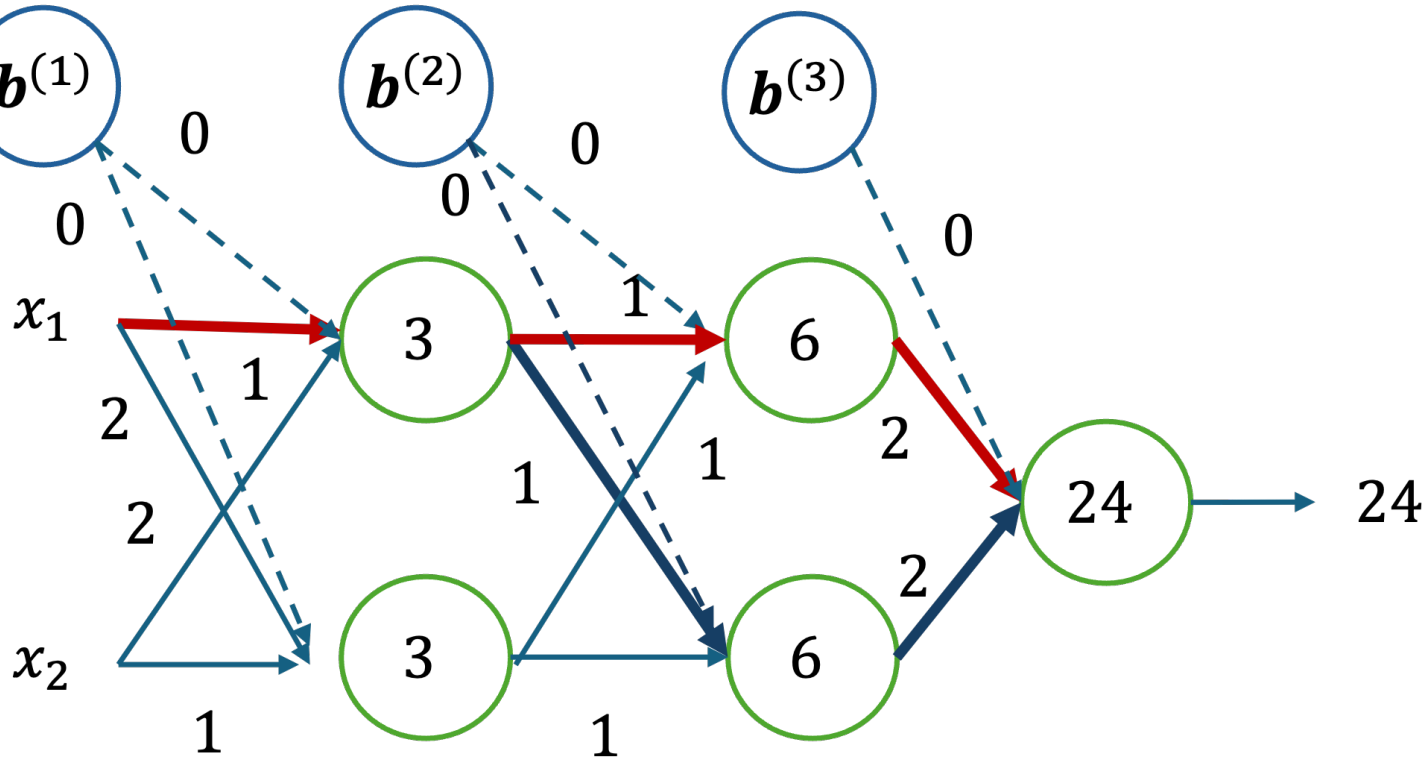
$$\mathbf{w}^{(3)} = \begin{bmatrix} w_{11}^{(3)} \\ w_{21}^{(3)} \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \mathbf{b}^{(1)} = \mathbf{b}^{(2)} = \mathbf{b}^{(3)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathbf{x} = [x_1, x_2]^T = [1, 1]^T,$$

$y = 24.1$.

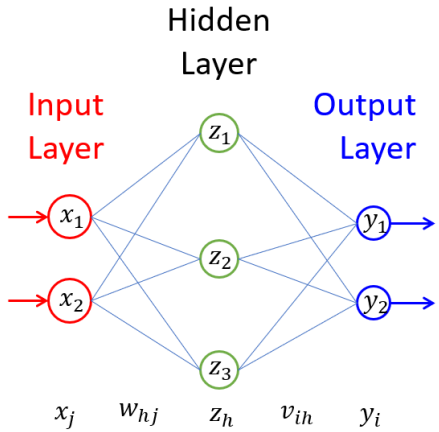
Chain Rule for Gradients (Cont'd)

By substituting the above values into Eq. 48, we can obtain the following expression:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = -(24 - 24.1)(2 \times 1 + 2 \times 1) \times 1 = 0.4 \tag{49}$$



Backpropagation Learning Algorithm



- Extension of the perceptron learning algorithm to multiple layers by error backpropagation from the outputs back to the inputs:
  - Learning of hidden-to-output weights: Like perceptron learning by treating the hidden units as inputs.
  - Learning of input-to-hidden weights: Applying the chain rule to calculate the gradient:

$$\frac{\partial E}{\partial w_{hj}} = \sum_i \frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial z_h} \frac{\partial z_h}{\partial w_{hj}} \tag{50}$$

MLP Learning for Nonlinear Regression

- Assuming a single output:

$$\hat{y}^{(n)} = \sum_{h=1}^{H} v_h z_h^{(n)} + v_0 \tag{51}$$

where $z_h^{(n)} = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x}^{(n)})$.

- Error function over entire training sample:

$$E(\mathbf{W}, \mathbf{v} | \mathcal{X}) = \frac{1}{2} \sum_{n=1}^{N} (y^{(n)} - \hat{y}^{(n)})^2 \tag{52}$$

- Update rule for second-layer weights:

$$v_h \leftarrow v_h - \eta \Delta v_h = v_h + \eta \sum_{n=1}^{N} \left(y^{(n)} - \hat{y}^{(n)} \right) z_h^{(n)} \tag{53}$$

MLP Learning for Nonlinear Regression (2)

- Update rule for first-layer weights:

$$
\begin{aligned}
w_{hj} &\leftarrow w_{hj} - \eta \frac{\partial E}{\partial w_{hj}} \\
&= w_{hj} - \eta \sum_{n=1}^{N} \frac{\partial E}{\partial \hat{y}^{(n)}} \frac{\partial \hat{y}^{(n)}}{\partial z_h^{(n)}} \frac{\partial z_h^{(n)}}{\partial w_{hj}} \\
&= w_{hj} + \eta \sum_{n=1}^{N} \left(y^{(n)} - \hat{y}^{(n)} \right) v_h z_h^{(n)} (1 - z_h^{(n)}) x_j^{(n)}
\end{aligned} \tag{54}
$$

- where $\left(y^{(n)} - \hat{y}^{(n)} \right) v_h$ acts like the error term for hidden unit h, which is backpropagated from the output to the hidden unit
- $\frac{\partial z_h^{(n)}}{\partial w_{hj}} = z_h^{(n)} (1 - z_h^{(n)}) x_j^{(n)}$ is for the sigmoid function but may have other forms for other activation functions.

MLP Learning for Nonlinear Regression (3)

- Online update rule of SGD for first and second-layer weights:

$$v_h \leftarrow v_h + \eta \left(y^{(n)} - \hat{y}^{(n)} \right) z_h^{(n)} \tag{55}$$

$$w_{hj} \leftarrow w_{hj} + \eta \left(y^{(n)} - \hat{y}^{(n)} \right) v_h z_h^{(n)} (1 - z_h^{(n)}) x_j^{(n)} \tag{56}$$

- Update rule of minibatch SGD for first and second-layer weights:

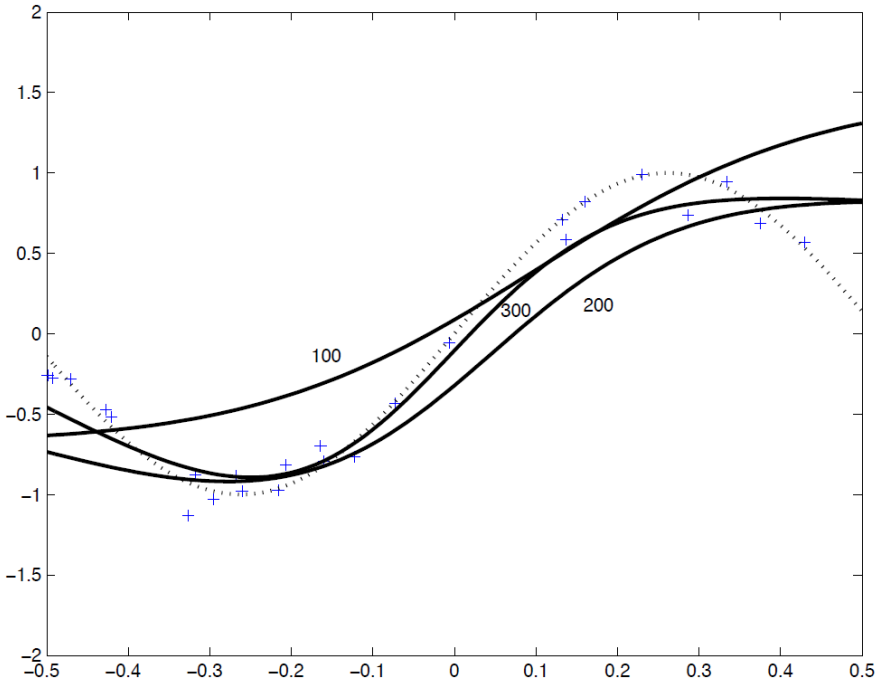
$$v_h \leftarrow v_h + \frac{\eta}{B} \sum_{n=1}^{B} \left(y^{(n)} - \hat{y}^{(n)} \right) z_h^{(n)} \tag{57}$$

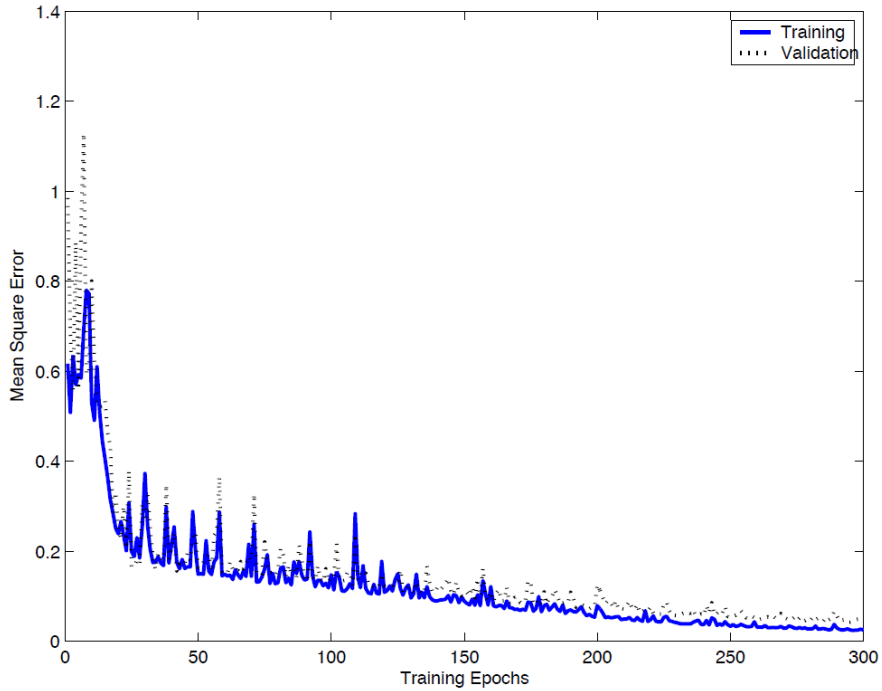
$$w_{hj} \leftarrow w_{hj} + \frac{\eta}{B} \sum_{n=1}^{B} \left(y^{(n)} - \hat{y}^{(n)} \right) v_h z_h^{(n)} (1 - z_h^{(n)}) x_j^{(n)} \tag{58}$$

where B is the size of the mini batch.

Example

Evolution of regression function and error over time





MLP Learning for Binary Classification

- A sigmoid output unit:

$$\hat{y}^{(n)} = \text{sigmoid}(\sum_{h=1}^{H} v_h z_h^{(n)} + v_0) \tag{59}$$

where $z_h^{(n)} = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x}^{(n)})$.

- Cross-entropy error function over the whole training sample:

$$E(\mathbf{W}, \mathbf{v}|\mathcal{X}) = -\frac{1}{2} \sum_{n=1}^{N} \left[y^{(n)} \log \hat{y}^{(n)} + (1 - y^{(n)}) \log(1 - \hat{y}^{(n)}) \right] \tag{60}$$

- Update rules for second-layer weights:

$$v_h \leftarrow v_h + \eta \sum_{n=1}^{N} \left(y^{(n)} - \hat{y}^{(n)} \right) z_h^{(n)} \tag{61}$$

MLP Learning for Binary Classification (2)

- Update rules for first-layer weights:

$$
\begin{aligned}
w_{hj} &\leftarrow w_{hj} - \eta \frac{\partial E}{\partial w_{hj}} \\
&= w_{hj} - \eta \sum_{n=1}^{N} \frac{\partial E}{\partial \hat{y}^{(n)}} \frac{\partial \hat{y}^{(n)}}{\partial z_h^{(n)}} \frac{\partial z_h^{(n)}}{\partial w_{hj}} \\
&= w_{hj} + \eta \sum_{n=1}^{N} \left(\frac{y^{(n)}}{\hat{y}^{(n)}} - \frac{1 - y^{(n)}}{1 - \hat{y}^{(n)}} \right) \hat{y}^{(n)} (1 - \hat{y}^{(n)}) v_h z_h^{(n)} (1 - z_h^{(n)}) x_j^{(n)} \\
&= w_{hj} + \eta \sum_{n=1}^{N} \left(y^{(n)} - \hat{y}^{(n)} \right) v_h z_h^{(n)} (1 - z_h^{(n)}) x_j^{(n)}
\end{aligned}
\tag{62}
$$

- Stochastic gradient descent or minibatch SGD can also be carried out.

MLP Learning for Multi-class Classification

- Output is a softmax function:

$$\hat{y}_i^{(n)} = \frac{\exp\left(o_i^{(n)}\right)}{\sum_{k=1}^{K} \exp\left(o_k^{(n)}\right)} \tag{63}$$

where $o_i^{(n)} = \sum_{h=1}^{H} v_{ih} z_h^{(n)} + v_{i0}$ and $z_h^{(n)} = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x}^{(n)})$.

- Error function:

$$E(\mathbf{W}, \mathbf{V} | \mathcal{X}) = -\sum_{i=1}^{K} \sum_{n=1}^{N} y_i^{(n)} \log \hat{y}_i^{(n)} \tag{64}$$

- Update rules for second-layer weights:

$$v_{ih} \leftarrow v_{ih} + \eta \sum_{n=1}^{N} \left(y_i^{(n)} - \hat{y}_i^{(n)}\right) z_h^{(n)} \tag{65}$$

MLP Learning for Multi-class Classification (2)

- Update rules for first-layer weights:

$$
\begin{aligned}
w_{hj} &\leftarrow w_{hj} - \eta \frac{\partial E}{\partial w_{hj}} \\
&= w_{hj} - \eta \sum_{n=1}^{N} \sum_{i=1}^{K} \frac{\partial E}{\partial \hat{y}_i^{(n)}} \frac{\partial \hat{y}_i^{(n)}}{\partial z_h^{(n)}} \frac{\partial z_h^{(n)}}{\partial w_{hj}} \\
&= w_{hj} + \eta \sum_{n=1}^{N} \left[\sum_{i=1}^{K} \left(y_i^{(n)} - \hat{y}_i^{(n)} \right) v_{ih} \right] z_h^{(n)} (1 - z_h^{(n)}) x_j^{(n)}
\end{aligned}
\tag{66}
$$

- Stochastic gradient descent or minibatch SGD can also be carried out in the same way.

An example

Consider the ridership of a public transportation system in the San Francisco bay area, Bay Area Rapid Transit (BART). Recently, a group of researchers is assuming that the ridership of the transit system is related to some socio-economic factors, which are preliminarily **the total population near each station, number of households that own 0 vehicles, total employment, and total road network density**. The raw census data is collected by the Environment Protection Agency of the United States[1] while the ridership data is available at the BART's official website[2]. The data for the catchment area for each station is processed by Github user HaTT2018[3].

Similarly, we can try to fit our multi-layer perceptron model to investigate how these four factors influence the ridership.

[1] https://www.epa.gov/sites/default/files/2021-06/documents/epa_sld_3.0_technicaldocumentationuserguide_may2021.pdf

[2] https://www.bart.gov/about/reports/ridership

[3] https://github.com/HaTT2018/BART_Ridership_Prediction

An example

To make this problem as simple as possible, we only investigate the ridership from station *Embarcadero* (the CBD of San Francisco) to other stations.



Data

The Jupyter Notebook file is available on GitHub.

Data

```
# take the first 48 stations becasue the
# last 2 stations were not yet opened
# by the time this data was collected
data_BART_sld = pd.read_csv('data_X.csv').iloc[:48, 1:]
data_BART_sld.head()
```

	TotPop	AutoOwn0	TotEmp	Tot Rd Dens
0	11084	1664	33120	42.232657
1	142973	20505	103479	32.296762
2	36177	4704	47570	31.822631
3	104191	8018	39663	32.894203
4	70630	1142	13924	20.531431

```
# The file "3d_daily.npy" contains the daily ridership
# data from each station to all the other stations.
# Its shape is [50, 50, 365]. Each dimension represents
# origin, destination, day in the year respectively

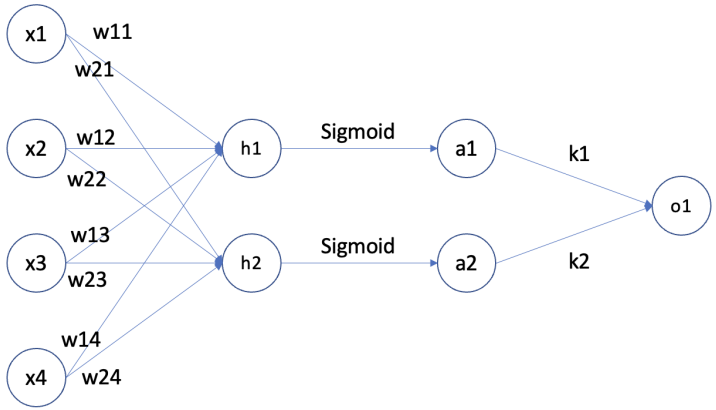
# Embarcadero is the 18th station in the list
OD_BART = np.load('3d_daily.npy').sum(axis=2)[17, :48]
```

```
print('Shape of data_BART_sld', data_BART_sld.shape)
print('Shape of OD_BART', OD_BART.shape)
```

```
Shape of data_BART_sld (48, 4)
Shape of OD_BART (48,)
```

Model Layout

Our job is to train an MLP model which takes "data_BART_sld" as input and generate "OD_BART" as output and make it as close to the real data as possible.
Consider a simple MLP, with 1 input layer, 1 hidden layer, and 1 output layer. The input layer has a dimension of 4 because we have 4 attributes of the scio-economic census. The output layer is 1-dimensional because the ridership is a scalar. The number of dimension of the hidden layer is set to be 2 for the sake of simplicity. The activation function is Sigmoid function as introduced in the lecture.



Forward Propagation

1st layer, input layer

$\mathbf{x} = [x_i]^\top, i \in \{1, 2, 3, 4\}$

2nd layer, hidden layer

$\mathbf{h} = [h_j]^\top, j \in \{1, 2\}$

$h_1 = x_1 \times w_{11} + x_2 \times w_{12} + x_3 \times w_{13} + x_4 \times w_{14}$

$h_2 = x_1 \times w_{21} + x_2 \times w_{22} + x_3 \times w_{23} + x_4 \times w_{24}$

2nd layer, hidden layer, activation

$\mathbf{a} = [a_r]^\top, r \in \{1, 2\}, a_1 = \sigma(h_1), a_2 = \sigma(h_2)$, where $\sigma(\cdot)$ is the sigmoid function.

3rd layer, output layer

$\mathbf{o} = o = k_1 \times a_1 + k_2 \times a_2$

Forward Propagation Cont'd

Given that $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$, $\mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}$, $\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$, $\mathbf{W}_{[j\times i]} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \end{bmatrix}$, $\mathbf{K}_{[1\times r]} = \begin{bmatrix} k_1 & k_2 \end{bmatrix}$, we can write the matrix form of forward propagation for more convenient code implementation.

2nd layer: $\mathbf{h} = \mathbf{W} \times \mathbf{x}$.
2nd layer activation: $\mathbf{a} = \sigma(\mathbf{h})$.
3rd layer: $\mathbf{o} = \mathbf{K} \times \mathbf{a}$.

* Recall that $\sigma(x) = \frac{1}{1+e^{-x}}$.

Gradient Descent and Loss Function

Suppose the label true value is y given input $\mathbf{x} = [x_1, x_2, x_3, x_4]^\top$, and we use squared loss function $L = \frac{1}{2}(o - y)^2$, which represents how much the output is deviated from the true value.

$\mathbf{W}$ and $\mathbf{K}$ contain all the parameters (10 numbers in total) we need to update in the training process. Before training starts, we assign some random initial values to our parameters $\mathbf{W}$ and $\mathbf{K}$. During training, denote lr as the learning rate, we obtain $\frac{\partial L}{\partial \mathbf{W}}$ and $\frac{\partial L}{\partial \mathbf{K}}$, then we use $\mathbf{W}_i = \mathbf{W}_{i-1} - \frac{\partial L}{\partial \mathbf{W}} \times lr$ and $\mathbf{K}_i = \mathbf{K}_{i-1} - \frac{\partial L}{\partial \mathbf{K}} \times lr$ to update $\mathbf{W}$ and $\mathbf{K}$ in order to make loss L smaller and smaller.

This is the basic idea of gradient decent, which is a popular set of non-convex optimization methods used in machine learning especially in deep learning.

Back Propagation

How to obtain $\frac{\partial L}{\partial \mathbf{W}}$ and $\frac{\partial L}{\partial \mathbf{K}}$?

$$\frac{\partial L}{\partial \mathbf{W}} = \left[\left(\frac{\partial L}{\partial W} \right)_{ji} \right], \tag{67}$$

where,

$$\left(\frac{\partial L}{\partial W} \right)_{ji} = \frac{\partial L}{\partial w_{ji}} \tag{68}$$

$$= \frac{\partial L}{\partial o} \frac{\partial o}{\partial a_j} \frac{\partial a_j}{\partial h_j} \frac{\partial h_j}{\partial w_{ji}} \tag{69}$$

$$= (o - y) k_j \frac{\partial \sigma(h_j)}{h_j} x_i \tag{70}$$

$$= (o - y) k_j \frac{e^{-h_j}}{\left(e^{-h_j} + 1\right)^2} x_i \tag{71}$$

Back Propagation Cont'd

Take $\left(\frac{\partial L}{\partial W}\right)_{13}$ as an example, where $j = 1$ and $i = 3$

$$\begin{aligned}
\left(\frac{\partial L}{\partial W}\right)_{13} &= \frac{\partial L}{\partial w_{13}} \\
&= \frac{\partial L}{\partial o}\frac{\partial o}{\partial a_1}\frac{\partial a_1}{\partial h_1}\frac{\partial h_1}{\partial w_{13}} \\
&= (o-y)k_1\frac{\partial \sigma(h_1)}{h_1}x_3 \\
&= (o-y)k_1\frac{e^{-h_1}}{(e^{-h_1}+1)^2}x_3
\end{aligned}$$

Back Propagation Cont'd

$$\frac{\partial L}{\partial \mathbf{K}} = \left[\left(\frac{\partial L}{\partial K} \right)_{1r} \right], \tag{72}$$

where,

$$\left(\frac{\partial L}{\partial K} \right)_{1r} = \frac{\partial L}{\partial k_{1r}} \tag{73}$$

$$= \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_r} \tag{74}$$

$$= (o - y) a_r \tag{75}$$

Take $\left(\frac{\partial L}{\partial K}\right)_{12}$ as an example,

$$\left(\frac{\partial L}{\partial K} \right)_{12} = \frac{\partial L}{\partial k_{12}} = \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_2} = (o - y) a_2$$

Back Propagation Cont'd

Finally,

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{bmatrix} (o-y)k_1 \frac{\partial \sigma(h_1)}{\partial h_1} x_1 & (o-y)k_1 \frac{\partial \sigma(h_1)}{\partial h_1} x_2 & (o-y)k_1 \frac{\partial \sigma(h_1)}{\partial h_1} x_3 & (o-y)k_1 \frac{\partial \sigma(h_1)}{\partial h_1} x_4 \\ (o-y)k_2 \frac{\partial \sigma(h_2)}{\partial h_2} x_1 & (o-y)k_2 \frac{\partial \sigma(h_2)}{\partial h_2} x_2 & (o-y)k_2 \frac{\partial \sigma(h_2)}{\partial h_2} x_3 & (o-y)k_2 \frac{\partial \sigma(h_2)}{\partial h_2} x_4 \end{bmatrix}$$

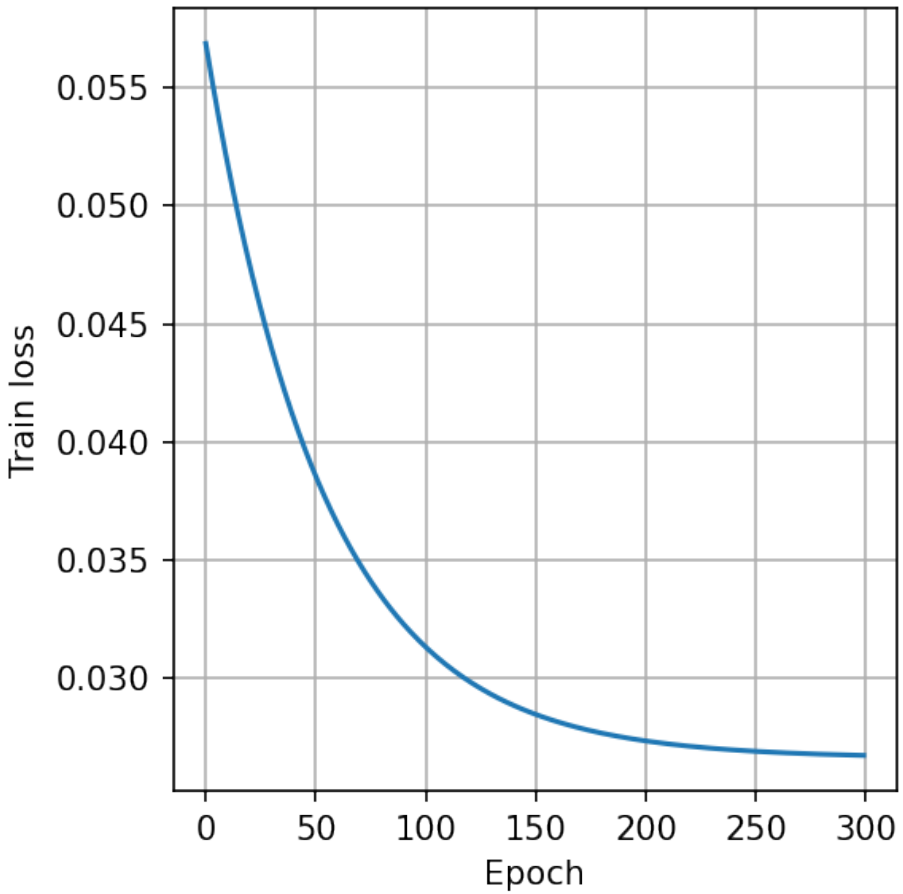
$$= (o-y) \begin{bmatrix} k_1 \frac{\partial \sigma(h_1)}{\partial h_1} \\ k_2 \frac{\partial \sigma(h_2)}{\partial h_2} \end{bmatrix} \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix}$$

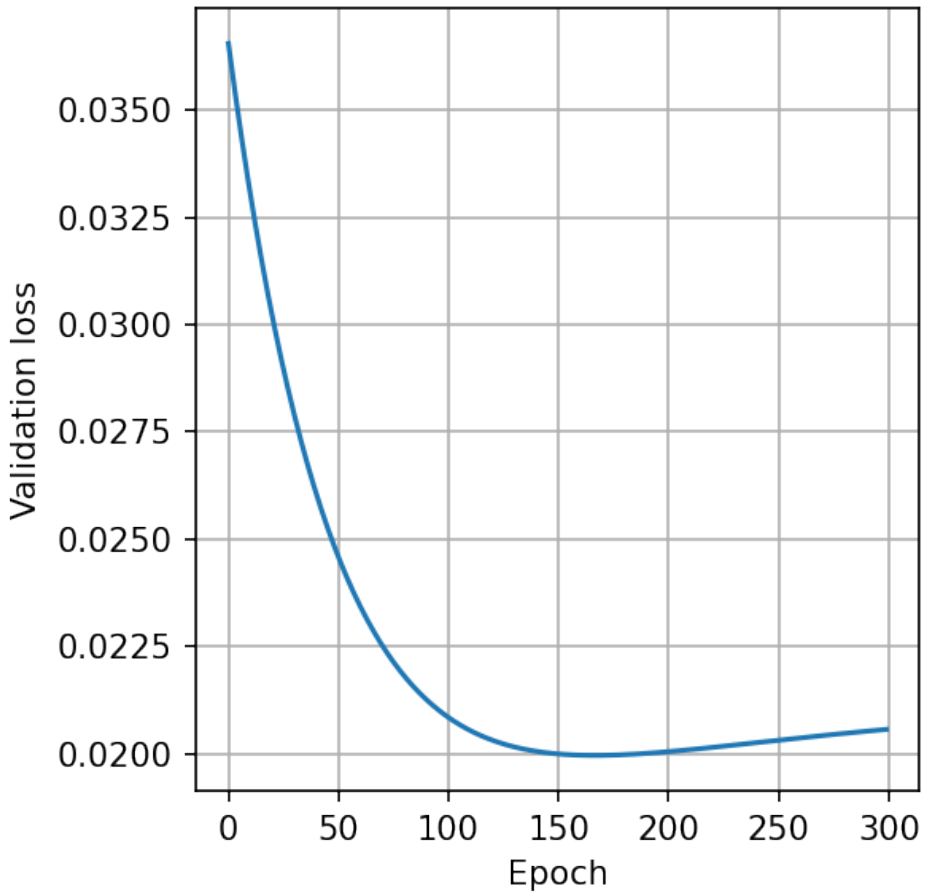
$$= \begin{bmatrix} k_1 & 0 \\ 0 & k_2 \end{bmatrix} \begin{bmatrix} \frac{\partial \sigma(h_1)}{\partial h_1} \\ \frac{\partial \sigma(h_2)}{\partial h_2} \end{bmatrix} (o-y) \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix}$$

$$\frac{\partial L}{\partial \mathbf{K}} = \begin{bmatrix} \frac{\partial L}{\partial k_1} & \frac{\partial L}{\partial k_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_1} & \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_2} \end{bmatrix} = \begin{bmatrix} (o-y)a_1 & (o-y)a_2 \end{bmatrix} = \left(\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} (o-y) \right)^\top =$$

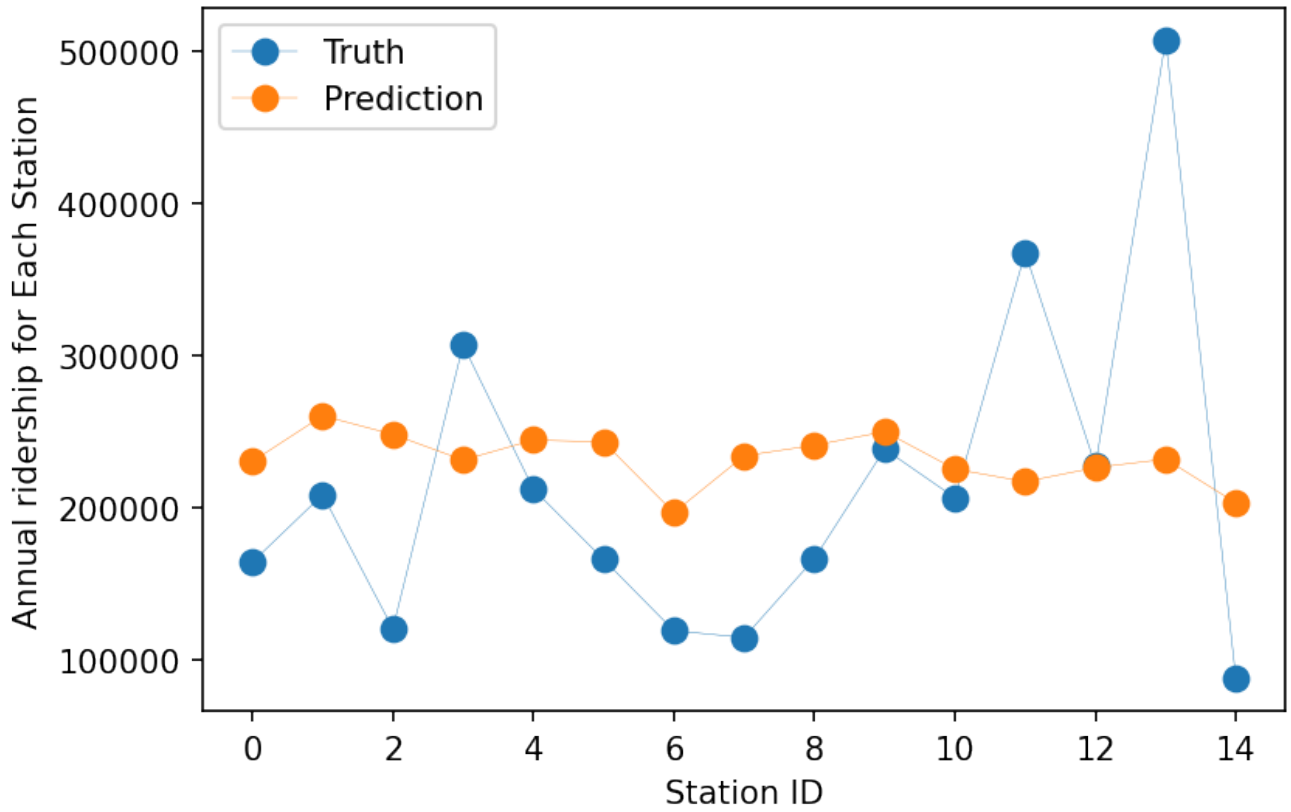
$$(o-y)^\top \mathbf{a}^\top$$

Loss


Train loss
0.055
0.050
0.045
0.040
0.035
0.030
0
50
100
150
200
250
300
Epoch


Validation loss
0.0350
0.0325
0.0300
0.0275
0.0250
0.0225
0.0200
0
50
100
150
200
250
300
Epoch

Prediction Results



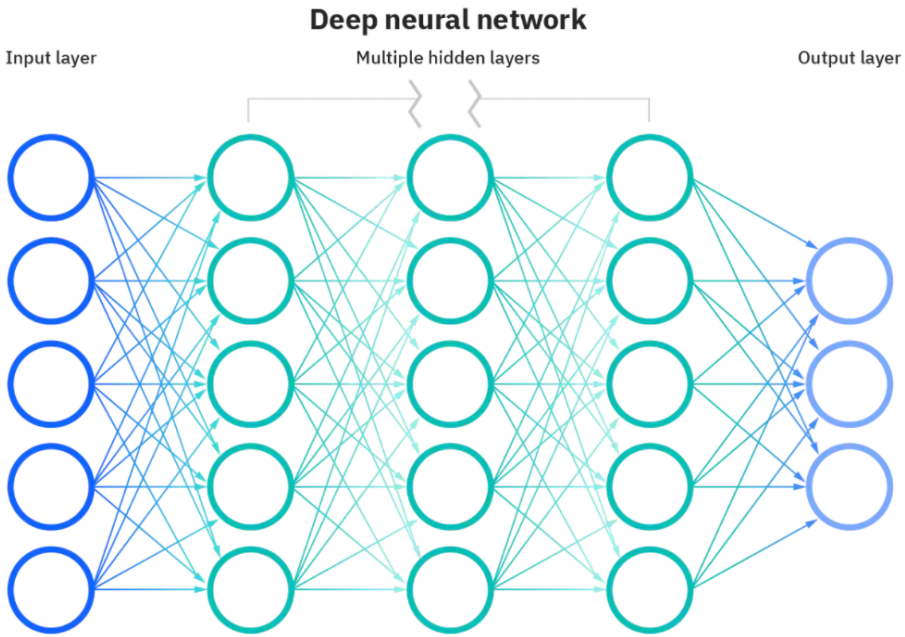
The results seem to be unsatisfactory, especially being malfunctioning on high ridership values, how to improve?

Neural Networks

- Basic Elements of Multilayer Perceptron (MLP)
  - Layers: input layer, output layer, hidden layers
  - Units (neurons): input units, output units, hidden units
  - Weights and Bias: parameters connecting units of two layers; for example, $z = \mathbf{W}^T\mathbf{x} + b$, where $\mathbf{W}$ are weights and b is bias.
  - Activation function: applies an element-wise nonlinear transformation of outputs of units; for example, $y = \text{sigmoid}(z) = \frac{1}{1+\exp(-z)}$
- Training of MLP: backpropagation with stochastic (minibatch) gradient descent.
- Problems to solve:
  - Regression: the single output unit is a linear function of hidden units
  - Classification: a softmax function is applied to the output layer

Deep Learning

- Normally, MLP has only one hidden layer, otherwise, the training is very difficult.
- Deep learning: neural networks with many hidden layers with various layer structures.



Outline

1. Introduction
2. Simple Perceptron
3. Multilayer Perceptron
4. Training Deep Neural Networks
   - Choosing Activation Functions
   - Initialization
   - Regularization
     - Early Stopping
   - Optimization Algorithms

Outline

1. Introduction
2. Simple Perceptron
3. Multilayer Perceptron
4. Training Deep Neural Networks
   - Choosing Activation Functions
   - Initialization
   - Regularization
     - Early Stopping
   - Optimization Algorithms

Gradient Vanishing & Gradient Exploding

- **Gradient Vanishing**: As the backpropagation algorithm advances backward from the output layer towards the input layer, the gradients often get smaller and smaller and approach zero, eventually leaving the weights of the initial or lower layers nearly unchanged. As a result, the gradient descent never converges to the optimum. This is known as the vanishing gradients problem.
- **Gradient Exploding**: The gradients keep getting larger in some cases as the backpropagation algorithm progresses. This, in turn, causes large weight updates and causes the gradient descent to diverge. This is known as the exploding gradient problem.

Challenges of Training Deep Neural Networks

- The vanishing gradient problem makes the lower layers (i.e., the hidden layers closer to the input) of a deep neural network very difficult to train, because the gradients often get smaller and smaller as the backpropagation (BP) learning algorithm progresses down to the lower layers. Similarly, the gradient exploding problem also makes the training very difficult.
- The many network weights in a deep neural network make it easy to overfit the training data.
- It would be extremely time-consuming to train a large network.

Sigmoid and Tanh Function

- The sigmoid function is defined as:

$$g(x) = \frac{1}{1 + \exp(-x)} \tag{76}$$

- The tanh function can also be used as the activation function:

$$g(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{77}$$

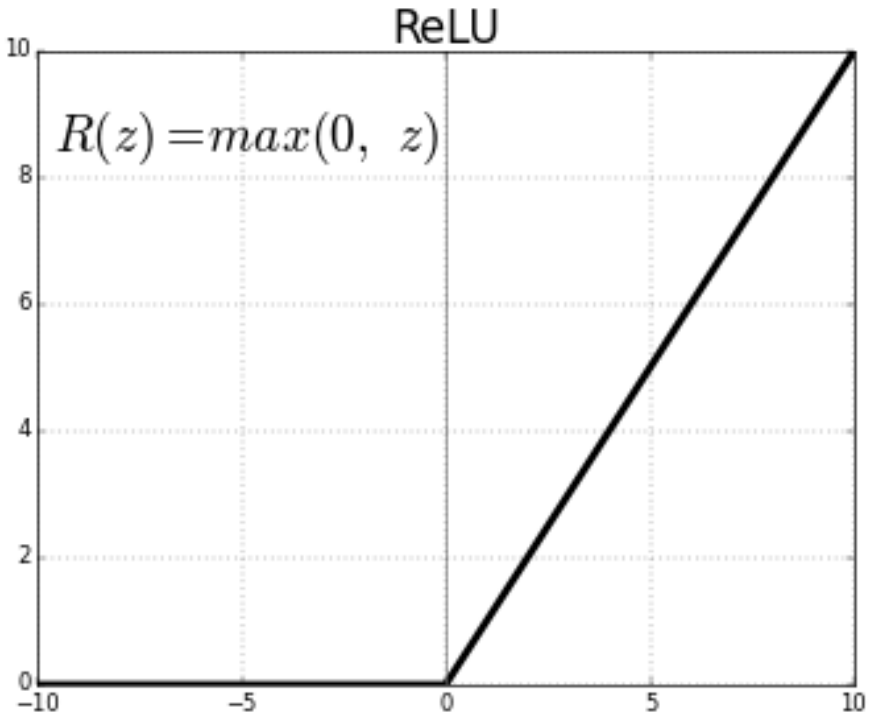
- Both sigmoid function and tanh function are differentiable
- The output of sigmoid function is in the range of $[0, 1]$ while that of tahn function is within $[-1, 1]$.

Rectified Linear Unit

- The rectifier is an activation function defined as:

$$g(x) = \max(0, x) \tag{78}$$

- A processing unit that uses the rectifier as activation function is called a rectifier linear unit (ReLU).



Rectified Linear Unit (2)

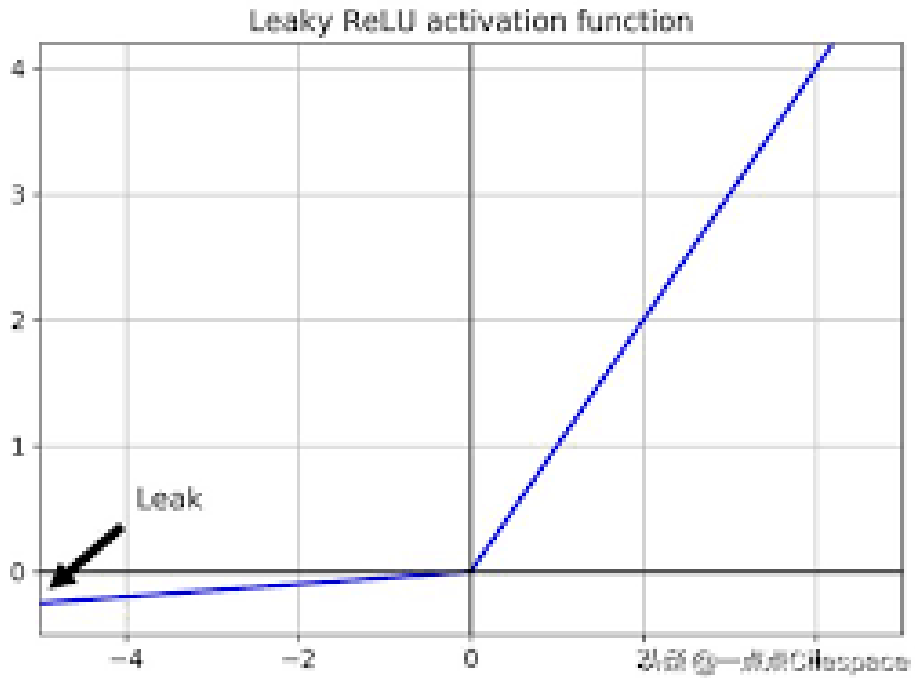
- ReLU alleviates the vanishing gradient problem because it does not saturate for positive input values.
- Another advantage of ReLU is that it is quite fast to compute.
- However, the vanishing gradient problem is not completely addressed because the gradient is 0 for negative input values. When this happens, the output is 0 and the gradient is 0, making the dying ReLU unlikely to come back to life again.
- Also, although the function is continuous, it is not differentiable when the input is 0. Nevertheless, this usually does not cause any problem in practice.

Leaky ReLU

- Leaky ReLU improves ReLU by allowing a small positive gradient for negative input values (e.g., $\alpha = 0.01$)::

$$g(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{otherwise} \end{cases} \tag{79}$$

where α is a hyperparameter (e.g., set as 0.0001).

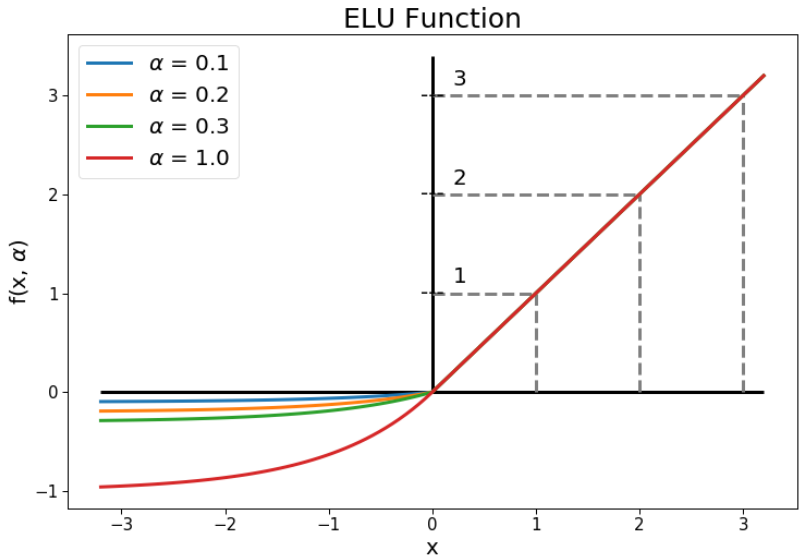


Exponential Linear Unit

- The exponential linear unit (ELU) has the following activation function:

$$g(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha(e^x - 1) & \text{otherwise} \end{cases} \tag{80}$$

where α is a hyperparameter (e.g., set as 1).



Exponential Linear Unit (2)

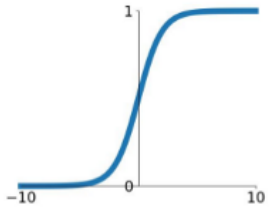
- An advantage of ELU is that its mean activation value is closer to 0, which can be proved to enable faster learning/convergence (less training epochs are required).
- Unlike ReLU and its variants, the activation function of ELU is smooth everywhere, including around x = 0, which speeds up gradient descent because it does not encounter non-differentiable problems at x = 0.
- The main drawback of the ELU activation function is that it is slower to compute than ReLU and its variants due to its use of the exponential function. This is not too much of a problem during training because the slower computation is compensated by the faster convergence, but during testing it will be considerably slower than ReLU.

Common Activation Functions

- Sigmoid, tanh, ReLU, Leaky ReLU, ELU

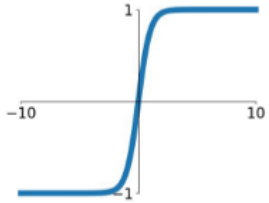
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



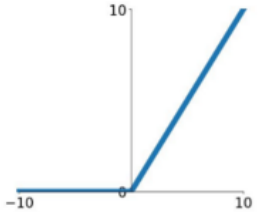
tanh

$$\tanh(x)$$



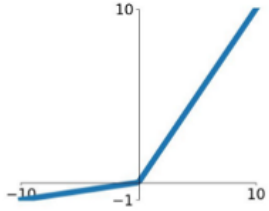
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

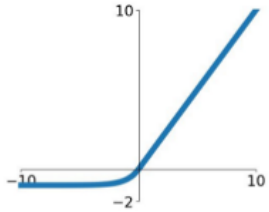


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

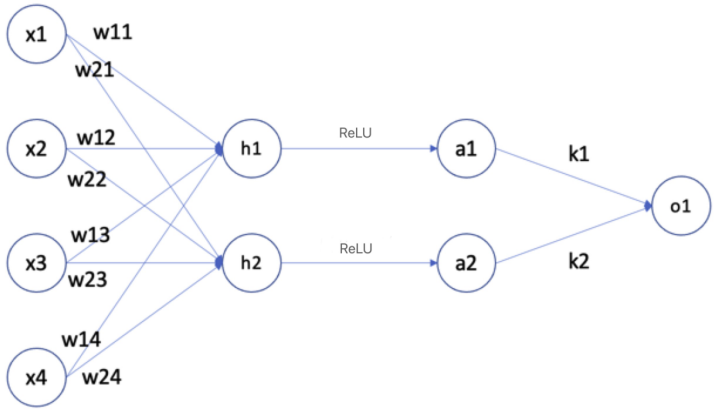


Which Activation Function to Choose?

- If runtime performance is not a concern, in general, the relative performance of different choices is:
  ELU > leaky ReLU (and its variants) > ReLU > tanh > sigmoid
- If runtime performance is an important factor to consider, leaky ReLU will be a better choice than ELU.
- The hyperparameters in leaky ReLU and ELU may be learned to further boost the performance if the training set is sufficiently large (or else overfitting may occur).

Example of ReLU (1)

Consider a simple MLP, with 1 input layer, 1 hidden layer, and 1 output layer. The input layer has a dimension of 4 because we have 4 attributes of the scio-economic census. The output layer is 1-dimensional because the ridership is a scalar. The number of dimensions of the hidden layer is set to 2 for the sake of simplicity. The activation function is the ReLU function as introduced in the lecture.



Forward Propagation

1st layer, input layer

$\mathbf{x} = [x_i]^\top, i \in \{1, 2, 3, 4\}$

2nd layer, hidden layer

$\mathbf{h} = [h_j]^\top, j \in \{1, 2\}$

$h_1 = x_1 \times w_{11} + x_2 \times w_{12} + x_3 \times w_{13} + x_4 \times w_{14}$

$h_2 = x_1 \times w_{21} + x_2 \times w_{22} + x_3 \times w_{23} + x_4 \times w_{24}$

2nd layer, hidden layer, activation

$\mathbf{a} = [a_r]^\top, r \in \{1, 2\}, a_1 = \text{ReLU}(h_1), a_2 = \text{ReLU}(h_2)$, where $\text{ReLU}(\cdot)$ is the ReLU function.

3rd layer, output layer

$\mathbf{o} = o = k_1 \times a_1 + k_2 \times a_2$

Forward Propagation Cont'd

Given that $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$, $\mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}$, $\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$, $\mathbf{W}_{[j\times i]} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \end{bmatrix}$, $\mathbf{K}_{[1\times r]} = \begin{bmatrix} k_1 & k_2 \end{bmatrix}$, we can write the matrix form of forward propagation for more convenient code implementation.

2nd layer: $\mathbf{h} = \mathbf{W} \times \mathbf{x}$.
2nd layer activation: $\mathbf{a} = \text{ReLU}(\mathbf{h})$.
3rd layer: $\mathbf{o} = \mathbf{K} \times \mathbf{a}$.

* Recall that $\text{ReLU}(x) = \max(0, x)$.

Gradient Descent and Loss Function

Suppose the label true value is y given input $\mathbf{x} = [x_1, x_2, x_3, x_4]^\top$, and we use squared loss function $L = \frac{1}{2}(o - y)^2$, which represents how much the output is deviated from the true value.

$\mathbf{W}$ and $\mathbf{K}$ contain all the parameters (10 numbers in total) we need to update in the training process. Before training starts, we assign some random initial values to our parameters $\mathbf{W}$ and $\mathbf{K}$. During training, denote lr as the learning rate, we obtain $\frac{\partial L}{\partial \mathbf{W}}$ and $\frac{\partial L}{\partial \mathbf{K}}$, then we use $\mathbf{W}_i = \mathbf{W}_{i-1} - \frac{\partial L}{\partial \mathbf{W}} \times lr$ and $\mathbf{K}_i = \mathbf{K}_{i-1} - \frac{\partial L}{\partial \mathbf{K}} \times lr$ to update $\mathbf{W}$ and $\mathbf{K}$ in order to make loss L smaller and smaller.

This is the basic idea of gradient decent, which is a popular set of non-convex optimization methods used in machine learning especially in deep learning.

Back Propagation

How to obtain $\frac{\partial L}{\partial \mathbf{W}}$ and $\frac{\partial L}{\partial \mathbf{K}}$?

$$\frac{\partial L}{\partial \mathbf{W}} = \left[\left(\frac{\partial L}{\partial W} \right)_{ji} \right], \tag{81}$$

where,

$$\left(\frac{\partial L}{\partial W} \right)_{ji} = \frac{\partial L}{\partial w_{ji}} \tag{82}$$

$$= \frac{\partial L}{\partial o} \frac{\partial o}{\partial a_j} \frac{\partial a_j}{\partial h_j} \frac{\partial h_j}{\partial w_{ji}} \tag{83}$$

$$= (o - y) k_j \frac{\partial \mathrm{ReLU}(h_j)}{h_j} x_i \tag{84}$$

$$\tag{85}$$

Back Propagation Cont'd

Take $\left(\frac{\partial L}{\partial W}\right)_{13}$ as an example, where $j = 1$ and $i = 3$

$$\begin{aligned}\left(\frac{\partial L}{\partial W}\right)_{13} &= \frac{\partial L}{\partial w_{13}} \\ &= \frac{\partial L}{\partial o}\frac{\partial o}{\partial a_1}\frac{\partial a_1}{\partial h_1}\frac{\partial h_1}{\partial w_{13}} \\ &= (o-y)k_1\frac{\partial \text{ReLU}(h_1)}{h_1}x_3\end{aligned}$$

Back Propagation Cont'd

$$\frac{\partial L}{\partial \mathbf{K}} = \left[\left(\frac{\partial L}{\partial K} \right)_{1r} \right], \tag{86}$$

where,

$$\left(\frac{\partial L}{\partial K} \right)_{1r} = \frac{\partial L}{\partial k_{1r}} \tag{87}$$

$$= \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_r} \tag{88}$$

$$= (o - y) a_r \tag{89}$$

Take $\left(\frac{\partial L}{\partial K}\right)_{12}$ as an example,

$$\left(\frac{\partial L}{\partial K} \right)_{12} = \frac{\partial L}{\partial k_{12}} = \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_2} = (o - y) a_2$$

Back Propagation Cont'd

Finally,

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{bmatrix} (o-y)k_1 \frac{\partial \text{ReLU}(h_1)}{\partial h_1} x_1 & (o-y)k_1 \frac{\partial \text{ReLU}(h_1)}{\partial h_1} x_2 & (o-y)k_1 \frac{\partial \text{ReLU}(h_1)}{\partial h_1} x_3 & (o-y)k_1 \frac{\partial \text{ReLU}(h_1)}{\partial h_1} x_4 \\ (o-y)k_2 \frac{\partial \text{ReLU}(h_2)}{\partial h_2} x_1 & (o-y)k_2 \frac{\partial \text{ReLU}(h_2)}{\partial h_2} x_2 & (o-y)k_2 \frac{\partial \text{ReLU}(h_2)}{\partial h_2} x_3 & (o-y)k_2 \frac{\partial \text{ReLU}(h_2)}{\partial h_2} x_4 \end{bmatrix}$$

$$= (o-y) \begin{bmatrix} k_1 \frac{\partial \text{ReLU}(h_1)}{\partial h_1} \\ k_2 \frac{\partial \text{ReLU}(h_2)}{\partial h_2} \end{bmatrix} \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix}$$

$$= \begin{bmatrix} k_1 & 0 \\ 0 & k_2 \end{bmatrix} \begin{bmatrix} \frac{\partial \text{ReLU}(h_1)}{\partial h_1} \\ \frac{\partial \text{ReLU}(h_2)}{\partial h_2} \end{bmatrix} (o-y) \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix}$$

$$\frac{\partial L}{\partial \mathbf{K}} = \begin{bmatrix} \frac{\partial L}{\partial k_1} & \frac{\partial L}{\partial k_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_1} & \frac{\partial L}{\partial o} \frac{\partial o}{\partial k_2} \end{bmatrix} = \begin{bmatrix} (o-y)a_1 & (o-y)a_2 \end{bmatrix} = \left(\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} (o-y) \right)^\top = (o-y)^\top \mathbf{a}^\top$$

ReLU Activation Function Gradient

Consider the Rectified Linear Unit (ReLU) function:

$$f(x) = \begin{cases} 0 & \text{if } x < 0, \\ x & \text{if } x \geq 0. \end{cases}$$

The gradient of the ReLU function can be defined as:

$$\frac{\partial \text{ReLU}(x)}{\partial x} = \begin{cases} 0 & \text{if } x < 0, \\ 1 & \text{if } x \geq 0. \end{cases}$$

Back Propagation Cont'd

- Suppose that $\mathbf{x} = [1, 1, 1, 1]$, $\mathbf{W}_{[j\times i]} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & -1 & -1 & -1 \end{bmatrix}$, $\mathbf{K} = [1, 1]^\top$, $y = 4.1$
- Forward Propagation: $h_1 = 1 \times 1 + 1 \times 1 + 1 \times 1 + 1 \times 1 = 4$, $h_2 = -1 \times 1 - 1 \times 1 - 1 \times 1 - 1 \times 1 = -4$, $a_1 = \text{ReLU}(h_1) = \text{ReLU}(4) = 4$, $a_2 = \text{ReLU}(h_2) = \text{ReLU}(-4) = 0$, $o = 1 \times a_1 + 1 \times a_2 = 4 + 0 = 4$.
- Back Propagation:

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{bmatrix} k_1 & 0 \\ 0 & k_2 \end{bmatrix} \begin{bmatrix} \frac{\partial \text{ReLU}(h_1)}{\partial h_1} \\ \frac{\partial \text{ReLU}(h_2)}{\partial h_2} \end{bmatrix} (o - y) \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix} =$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} (4 - 4.1) \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} -0.1 & -0.1 & -0.1 & -0.1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Outline

1. Introduction
2. Simple Perceptron
3. Multilayer Perceptron
4. Training Deep Neural Networks
   - Choosing Activation Functions
   - Initialization
   - Regularization
     - Early Stopping
   - Optimization Algorithms

Xavier Initialization

- When training the neural networks, the parameters (weights) need to be initialized.
- Observations:
  - If the weights in a network start too small, then the signal shrinks as it passes through each layer until it is too tiny to be useful.
  - If the weights in a network start too large, then the signal grows as it passes through each layer until it is too massive to be useful.
- The weight initialization strategy called Xavier initialization (named after the author's first name) makes sure the weights are in a reasonable range of values so that the signals can reach deep into the network.
- The major idea is to let the distribution governing the initialization of weights depend on the number of input connections and the number of output connections for the layer whose weights are being initialized.

Xavier Initialization for Sigmoid Function

- Let n_{in} and n_{out} denote the number of input connections and number of output connections, respectively, in a layer.
- The weights may be initialized randomly using a normal distribution or a uniform distribution.
- Normal distribution with zero mean and variance

$$\sigma^2 = \frac{2}{n_{in} + n_{out}} \tag{90}$$

- Uniform distribution in $[-r, +r]$ where

$$r = \sqrt{\frac{6}{n_{in} + n_{out}}} \tag{91}$$

He Initialization

- The He initialization (named after the author's last name) is for ReLU, ELU and their variants.
- Normal distribution with zero mean and variance

$$\sigma^2 = \frac{4}{n_{in} + n_{out}} \tag{92}$$

- Uniform distribution in $[-r, +r]$ where

$$r = \sqrt{\frac{12}{n_{in} + n_{out}}} \tag{93}$$

Batch Normalization

- Although using a carefully designed weight initialization strategy along with a non-saturating activation function such as leaky ReLU or ELU can significantly alleviate the vanishing gradient problem at the beginning of training, this problem may still arise later during training.
- Batch normalization is a technique for addressing the vanishing/exploding gradient problem by addressing the more general internal covariate shift problem (i.e., the distribution of the inputs of each layer changes during training as the network weights of the previous layers change).
- Before applying the activation function of each layer, an operation involving scaling and shifting is added to zero-center and normalize the inputs.
- For more details, please refer to this useful tutorial.

Batch Normalization (2)

- The mean and standard deviation of the inputs over the current mini-batch are evaluated for normalizing the inputs during training (hence the name "batch normalization").
- With batch normalization, the vanishing gradient problem can be significantly reduced even when saturating activation functions such as the sigmoid and tangent functions are used, without relying on carefully designed weight initialization and regularization.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

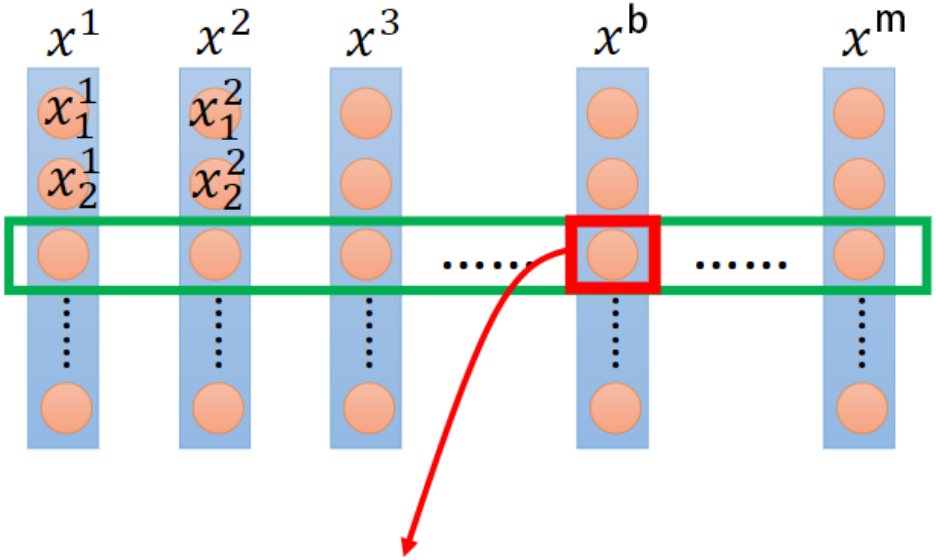
$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

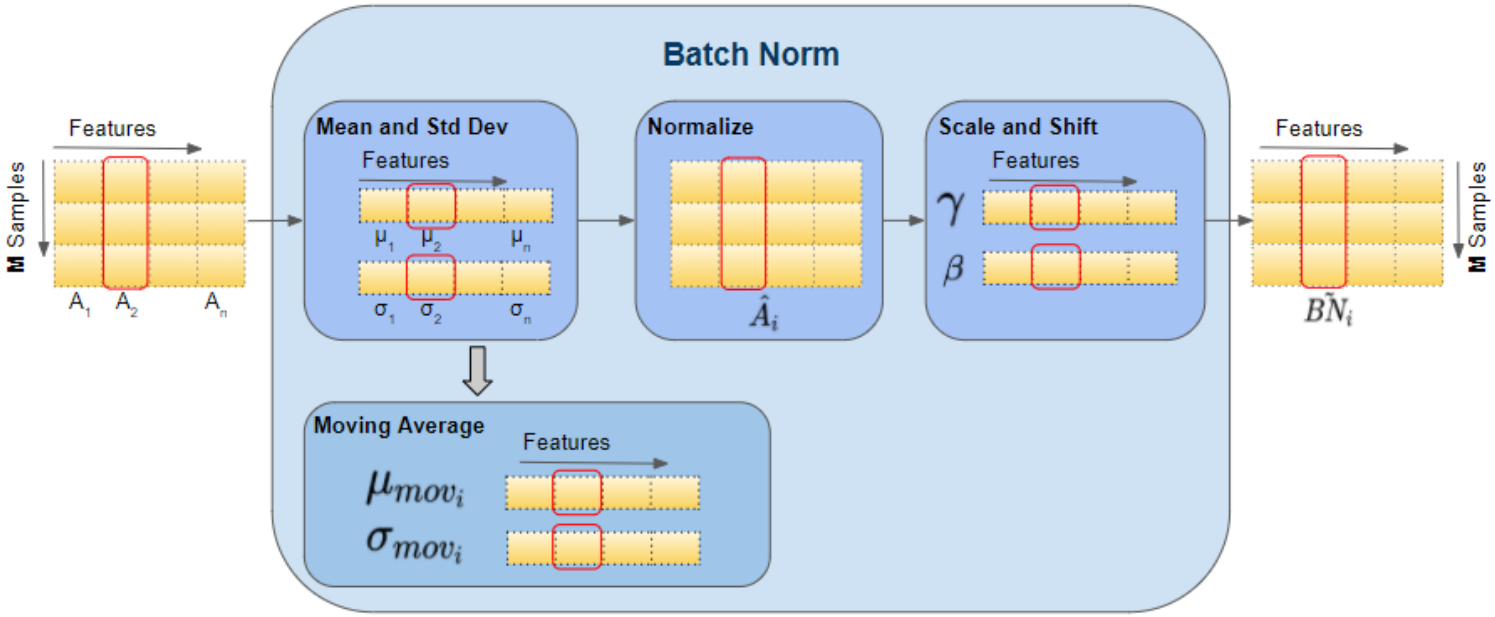
Batch Normalization (3)



$$x_1^{1-m} \sim \mathcal{N}(\mu_1, \sigma_1^2) \Rightarrow \hat{x}_1^{1-m} \sim \mathcal{N}(0,1) \Rightarrow y_1^{1-m} \sim \mathcal{N}(\beta, \gamma^2)$$
$$x_2^{1-m} \sim \mathcal{N}(\mu_2, \sigma_2^2) \Rightarrow \hat{x}_2^{1-m} \sim \mathcal{N}(0,1) \Rightarrow y_2^{1-m} \sim \mathcal{N}(\beta, \gamma^2)$$
$$\vdots$$

Batch Normalization (4)

- Two new parameters, one for scaling γ and the other for shifting β, are learned for each layer.
- At test time, the mean and standard deviation of the whole training set (computed efficiently using a moving average during training) are used instead.



Batch Normalization (5)

- Batch Normalization is generally placed **after the layer** (e.g., linear, convolutional) and **before the activation function**.
  - Linear Layer $\rightarrow$ BatchNorm $\rightarrow$ Activation (e.g., ReLU, Sigmoid).
  - Conv2D Layer $\rightarrow$ BatchNorm $\rightarrow$ Activation (e.g., ReLU).
- In residual blocks, Batch Normalization is applied before combining with the skip connection.
  - Conv2D $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv2D $\rightarrow$ BatchNorm $\rightarrow$ Addition (Skip Connection) $\rightarrow$ ReLU.
- Avoid applying Batch Normalization after Dropout.
  - In simple networks (like MLP), Batch Normalization is placed before Dropout.
  - In deeper networks (such as ResNet), Batch Normalization is enough to prevent overfitting, so only Batch Normalization is used.

Outline

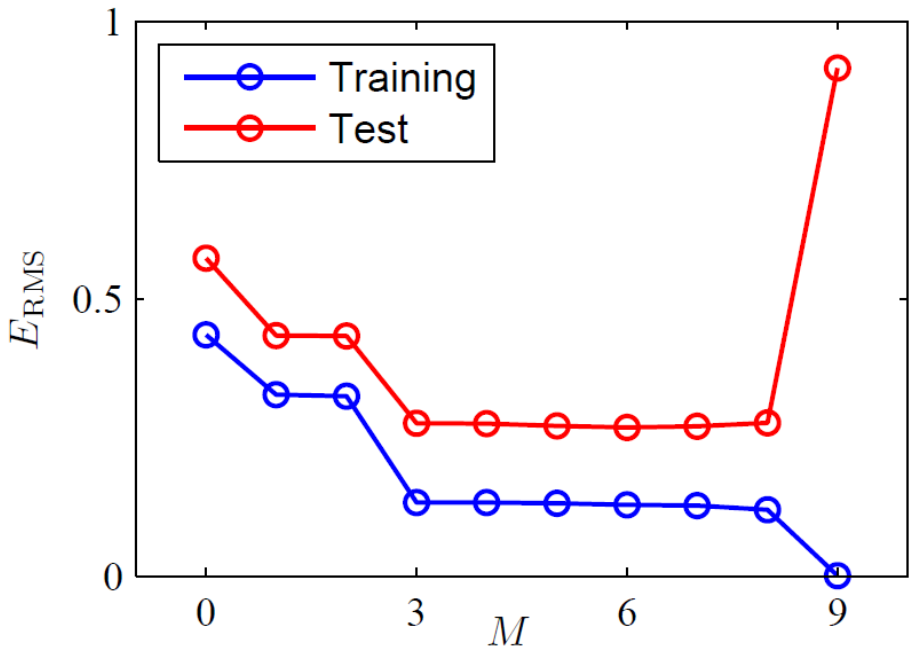
1. Introduction
2. Simple Perceptron
3. Multilayer Perceptron
4. Training Deep Neural Networks
   - Choosing Activation Functions
   - Initialization
   - Regularization
     - Early Stopping
   - Optimization Algorithms

Review Some Concepts

- Training error: error on training set
- Generalization error or test error: error on test set.
- Capacity of model: Ability of fitting a wide variety of functions.
  - In general, the capacity of a neural network increases with the number of parameters.

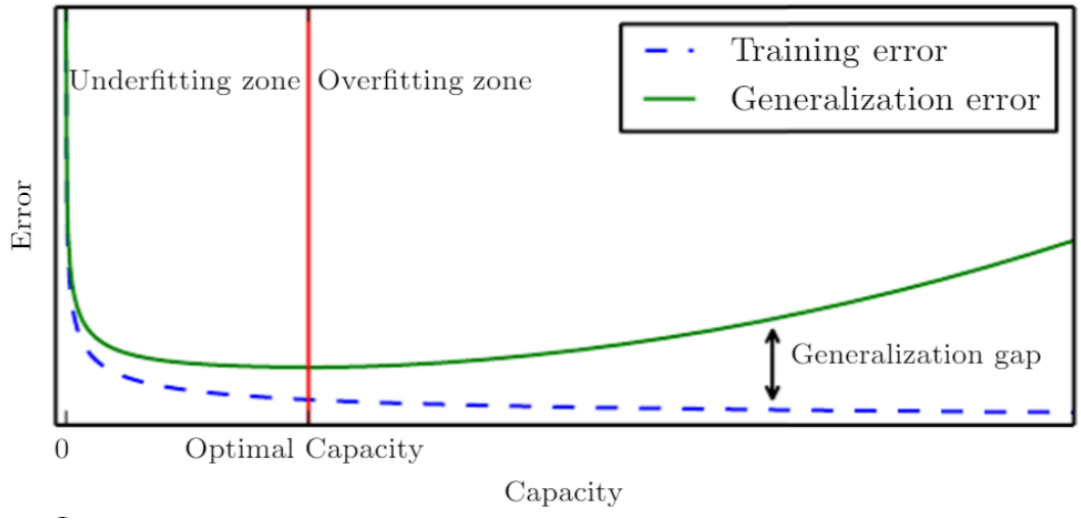
Revisit of Over-fitting Issue

- A too complex model, such as a linear model with too complex nonlinear basis functions, can lead to over-fitting issue, namely, having good performance in the training set but bad performance in the testing set.
- Two methods to address over-fitting issue:
  - Increase the number of samples.
  - Shrinkage Method: adding a regularization term to the error function.



Regularization in Deep Learning

- Model training aims at minimizing training error (loss), but our real objective is to minimize test error.
- While training error decreases with model capacity until reaching minimum, test error has a U-shaped curve as a function of model capacity.
- Regularization can reduce test error, possibly at the expense of training error.



Regularization in Deep Learning (2)

- Deep learning usually uses a model with more capacity than necessary.
- L2 regularization term is applied to the loss function:

$$\tilde{J}(\mathbf{w}) = J(\mathbf{w}) + \lambda \|\mathbf{w}\|^2 \tag{94}$$

  which is called L2 Penalty or Ridge regression.
- L1 regularization term:

$$\tilde{J}(\mathbf{w}) = J(\mathbf{w}) + \lambda |\mathbf{w}| \tag{95}$$

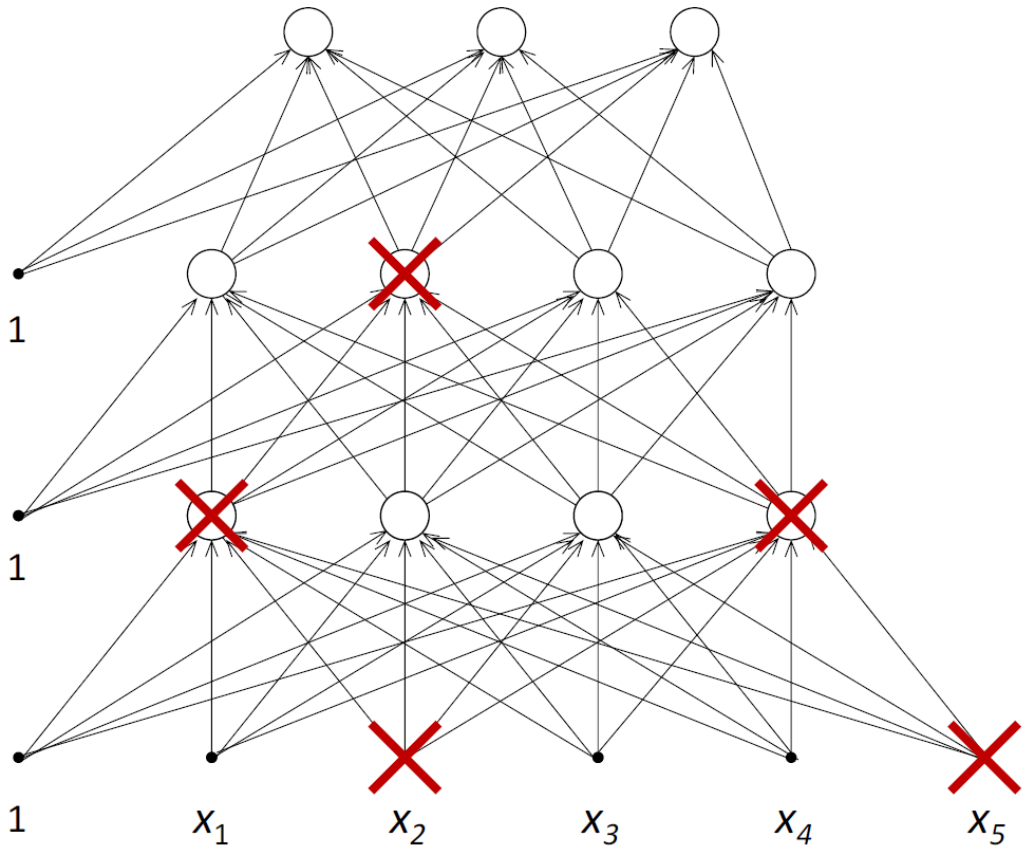
  which is called L1 Penalty or Lasso regression.
- The penalty pushes some of the parameters toward 0, and thereby determine the appropriate number of effective parameters.

Dropout

- Dropout is a regularization technique to prevent overfitting.
- At each training step, each hidden unit or input has a probability p of being temporarily dropped out (meaning that it will be entirely ignored during this training step, but it may become active again during the next step).
- The hyperparameter p, called the dropout rate, is typically set to 0.5 (but other values are also possible, even with different values for different layers). Sometimes it is more convenient to refer to the keep probability which is equal to $1 - p$.
- In general, a higher dropout rate is used if more serious overfitting is observed and for layers with more units (and hence more parameters to estimate).
- Dropout is only applied during training but not during testing.

Dropout (2)

- There is only one network, no subnetworks. At each step, training is conducted in a part of the network (a subnetwork).

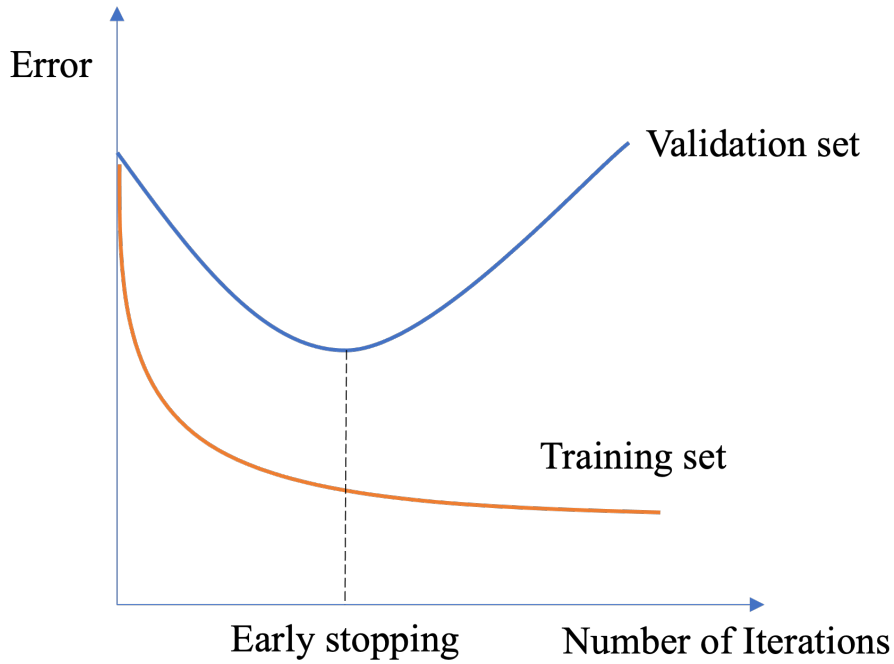


Why Does Dropout Work?

- Since hidden units and inputs are dropped out randomly, the network needs to learn to be more robust (i.e., less sensitive to slight changes in the inputs) by relying on more incoming units or inputs rather than just a few, effectively spreading out the weights to more units.
- An alternative way is to view dropout as ensemble learning, in that
  - each training step "generates" one of 2^N possible networks, where N is the total number of droppable units or inputs, and trains the network on one example from the training set.
  - The resulting neural network is essentially an ensemble of a large number of smaller networks.

Early Stopping

- Early Stopping: A form of regularization used to **prevent overfitting** in machine learning and deep learning models.
  - Monitor the model's performance on a validation set during the training process.
  - Stop training when the performance starts to degrade.



Outline

1. Introduction
2. Simple Perceptron
3. Multilayer Perceptron
4. Training Deep Neural Networks
   - Choosing Activation Functions
   - Initialization
   - Regularization
     - Early Stopping
   - Optimization Algorithms

Optimization in Training

- Our task is to find parameters (weights) $\mathbf{w}$ to minimize the loss function $J(\mathbf{w})$.
- There are multiple algorithms that we can use. Stochastic gradient descent is one of them.
  1. Initialize $\mathbf{w}$
  2. Repeat until convergence
     1. Sample a minibatch of m samples $((\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), ..., (\mathbf{x}^{(m)}, y^{(m)}))$ from the training set
     2. Compute the gradient: $\Delta\mathbf{w} \leftarrow \frac{1}{m}\sum_{n=1}^{m} \nabla_{\mathbf{w}} \mathcal{L}\left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)}\right)$
     3. Update parameters: $\mathbf{w} \leftarrow \mathbf{w} - \eta\Delta\mathbf{w}$, where η is the learning rate.
- There are others: momentum, AdaGrad, RMSProp, Adam .....

Momentum

- Momentum is a technique to accelerate SGD.
  1. Initialize $\mathbf{w}$
  2. Repeat until convergence
     1. Sample a minibatch of m samples $((\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)}))$ from the training set
     2. Compute the gradient: $\Delta\mathbf{w} \leftarrow \frac{1}{m}\sum_{n=1}^{m} \nabla_{\mathbf{w}} \mathcal{L}\left(f(\mathbf{x}^{(n)}, \mathbf{w}), y^{(n)}\right)$
     3. Compute velocity update: $\mathbf{v} \leftarrow \alpha\mathbf{v} - \eta\Delta\mathbf{w}$
     4. Update parameters: $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$
- In SGD, its movement is determined by gradient and learning rate.
- In SGD with momentum, the movement is additionally affected by its velocity $\mathbf{v}$, which is an exponentially decaying average of past negative gradients.

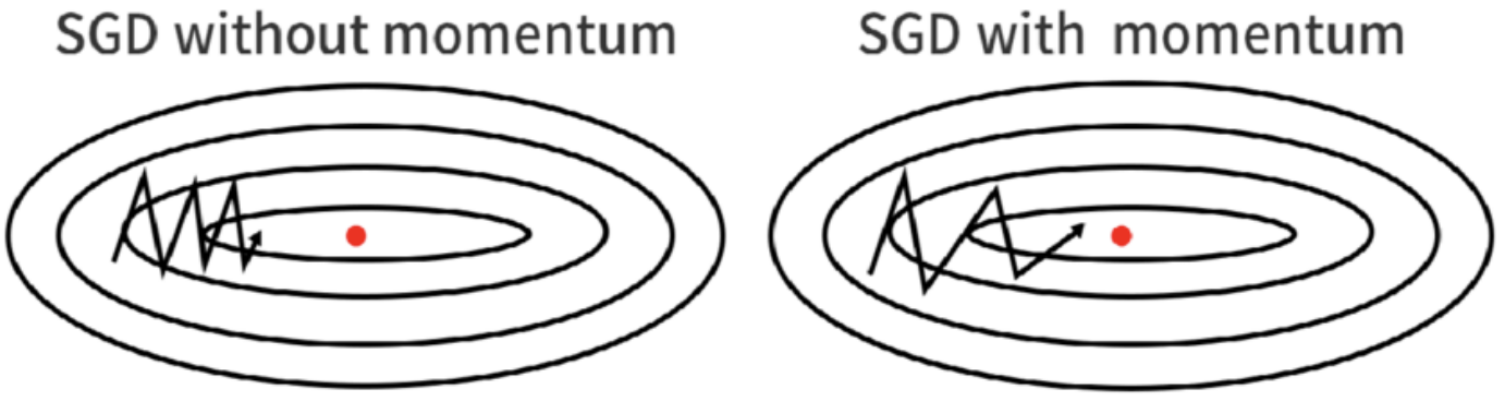
SGD with Momentum

- In SGD, its movement is determined by gradient and learning rate.
- In SGD with momentum, the movement is additionally affected by its velocity $\mathbf{v}$, which is an exponentially decaying average of past negative gradients.

$$
\begin{aligned}
\mathbf{v}^{(i+1)} &= \alpha \mathbf{v}^{(i)} - \eta \Delta \mathbf{w}^{(i)} \\
\mathbf{v}^{(i)} &= \alpha \mathbf{v}^{(i-1)} - \eta \Delta \mathbf{w}^{(i-1)} \\
\mathbf{v}^{(i-1)} &= \alpha \mathbf{v}^{(i-2)} - \eta \Delta \mathbf{w}^{(i-2)} \\
&\vdots \\
&\Downarrow \\
\mathbf{v}^{(i+1)} &= -\eta(\Delta \mathbf{w}^{(i)} + \alpha \Delta \mathbf{w}^{(i-1)} + \alpha^2 \Delta \mathbf{w}^{(i-2)} + \cdots) \\
\mathbf{w}^{(i+1)} &= \mathbf{w}^{(i)} + \mathbf{v}^{(i+1)}
\end{aligned}
$$

Momentum (2)

- The incorporation of momentum into stochastic gradient descent reduces the variation in overall gradient directions and speeds up learning.



AdaGrad

- AdaGrad improves stochastic gradient descent (SGD) using an adaptive learning rate which decays faster for steeper dimensions.
- SGD:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L \tag{96}$$

- AdaGrad:

$$\mathbf{s} \leftarrow \mathbf{s} + \nabla_{\mathbf{w}} L \circ \nabla_{\mathbf{w}} L \tag{97}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L \oslash \sqrt{\mathbf{s} + \epsilon} \tag{98}$$

where $\circ$ denotes the Hadamard product (a.k.a. elementwise multiplication), $\oslash$ denotes the Hadamard division (a.k.a. elementwise division), and ϵ a smoothing term with a small value (e.g., 10^{-10}) for all dimensions to avoid division by 0.

AdaGrad (2)

- The first step of AdaGrad accumulates the square of the gradients into the vector $\mathbf{s}$: a dimension of $\mathbf{s}$ will become larger and larger if the loss function is steep along that dimension.
- The second step is similar to SGD except that there is a scaling factor $\sqrt{\mathbf{s} + \epsilon}$: a dimension will decay its learning rate faster if the loss function is steep along that dimension.
- With the adaptive learning rate (without requiring manual tuning), AdaGrad helps point the resulting updates more directly towards the global optimum.
- Although AdaGrad performs well for simple quadratic problems, it often stops too early before reaching the global optimum when used for training neural networks.

RMSProp

- To overcome the problem of AdaGrad which decays too fast, RMSProp does not accumulate all the gradients from the beginning but only those from the recent iterations by introducing exponential decay in the first step.
- RMSProp:

$$\mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta) \nabla_{\mathbf{w}} L \circ \nabla_{\mathbf{w}} L \tag{99}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L \oslash \sqrt{\mathbf{s} + \epsilon} \tag{100}$$

where the decay rate β is typically set to 0.9.

Adam

- Adam, which stands for adaptive moment estimation, combines the ideas of momentum and RMSProp:
  - Like momentum, Adam keeps track of an exponentially decaying average of the past gradients
  - Like RMSProp, it keeps track of an exponentially decaying average of the past squared gradients.
- Adam:

$$\Delta \mathbf{w} \leftarrow \beta_1 \Delta \mathbf{w} - (1 - \beta_1) \nabla_{\mathbf{w}} L \tag{101}$$

$$\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\mathbf{w}} L \circ \nabla_{\mathbf{w}} L \tag{102}$$

$$\Delta \mathbf{w} \leftarrow \frac{\Delta \mathbf{w}}{1 - \beta_1^t} \tag{103}$$

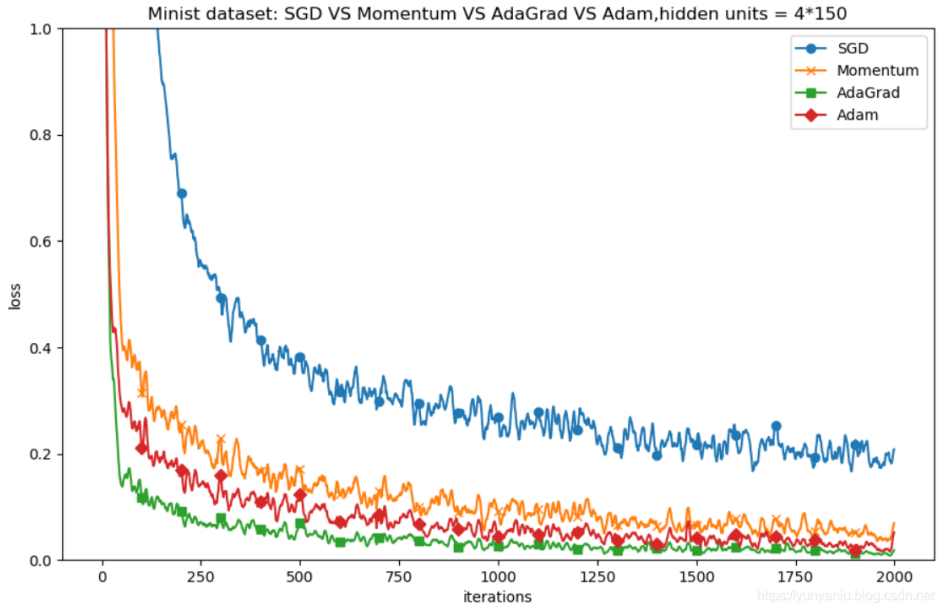
$$\mathbf{s} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t} \tag{104}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \Delta \mathbf{w} \oslash \sqrt{\mathbf{s} + \epsilon} \tag{105}$$

where t is the iteration number.

Performances of Optimizers

- Although Adam generally works well and is faster than other methods, a recent study showed that adaptive optimization methods including AdaGrad, RMSProp and Adam can give solutions that generalize poorly on some data sets.
- So there is no single method that is always the best.



How to Determine Learning Rate?

- Instead of using a fixed learning rate throughout the whole training process, different learning schedules can be used to reduce the learning rate during training.
- Some common learning schedules:
  - Predetermined piecewise constant learning rate
  - Exponential scheduling
  - Power scheduling
- Since AdaGrad, RMSProp and Adam automatically reduce the learning rate during training, there is no need to add an extra learning schedule for these optimizers.